

AUTOMATED DETECTION, TRACKING AND TACTICAL ANALYSIS IN BROADCAST FOOTBALL VIDEO

A REPORT SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF BACHELOR OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

Aryan Tiwari

Supervised by Dr Aphrodite Galata

Department of Computer Science

Contents

Abstract	10
Declaration	11
Copyright	12
Acknowledgements	13
1 Introduction	14
1.1 Motivation and Context	14
1.2 Aims and Objectives	15
1.3 Evaluation Strategy	16
1.4 Report Structure	17
2 Background and Theory	19
2.1 Broadcast Football Video Analysis	19
2.2 Related Work	20
2.2.1 Commercial Tracking Systems	20
2.2.2 Academic Benchmarks and Integrated Pipelines	20
2.2.3 Positioning of This Project	21
2.3 Machine Learning and Computer Vision Foundations	22
2.3.1 Supervised Learning for Visual Recognition	22
2.3.2 Neural Networks and Multi-Layer Perceptrons	23
2.3.3 Convolutional Neural Networks	24
2.3.4 Transfer Learning and Fine-Tuning	25
2.4 Object Detection in Broadcast Football	26
2.4.1 Bounding Boxes, IoU, and Non-Maximum Suppression	26
2.4.2 Precision, Recall, and Mean Average Precision	27

2.4.3	The YOLO Architecture	29
2.5	Multi-Object Tracking	30
2.5.1	Tracking-by-Detection	32
2.5.2	Motion Prediction and Data Association	32
2.5.3	ByteTrack	34
2.5.4	BoT-SORT as an Alternative Tracker	36
2.5.5	Tracking Evaluation: HOTA	36
2.6	Team Identification from Visual Features	37
2.6.1	Team Identification as Clustering	37
2.6.2	Visual Representations for Team Identification	37
2.6.3	Dimensionality Reduction and K-Means	38
2.6.4	Temporal Stability and Special Cases	39
2.7	Pitch Registration and Planar Homography	40
2.7.1	Pitch Keypoint Detection	40
2.7.2	Planar Homography	41
2.7.3	Reprojection Error and Bird's-Eye Mapping	42
2.8	Spatial and Tactical Analysis in Pitch Space	43
2.8.1	Player Projection onto the Pitch	43
2.8.2	Heatmap Construction	44
2.8.3	Speed and Distance Estimation	44
2.8.4	Possession and Pass Detection	45
2.8.5	Offside Rule Formalisation	46
2.9	Error Propagation in a Multi-Stage Pipeline	47
2.10	Chapter Summary	49
3	Methodology and Implementation	50
3.1	Chapter Overview	50
3.2	Requirements and Design Objectives	51
3.2.1	Functional Requirements	51
3.2.2	Non-functional Requirements	52
3.2.3	Success Criteria	54
3.3	System Architecture	55
3.3.1	Pipeline Overview	55
3.3.2	Data Flow and Intermediate Representations	57
3.3.3	System Outputs	60
3.4	Datasets and Model Training	61

3.4.1	Datasets	61
3.4.2	Generic Object Detector Training	62
3.5	Small-Object Detection and Dedicated Ball Detection	63
3.5.1	Why Small Objects Are Harder to Detect	63
3.5.2	Strategies for Small-Object Detection	64
3.5.3	Temporal Filtering of Ball Candidates	65
3.5.4	Dedicated Ball Detector Training	65
3.5.5	Pitch Keypoint Detector Training	66
3.6	Detection and Tracking	67
3.6.1	Main Object Detection	67
3.6.2	Dedicated Ball Detection and Filtering	67
3.6.3	Multi-object Tracking	68
3.6.4	Track Handling and Failure Cases	69
3.7	Team Identification	69
3.7.1	Warm-up Sampling and Player Cropping	69
3.7.2	Embedding and Clustering	70
3.7.3	Temporal Smoothing and Role Handling	70
3.8	Pitch Registration	71
3.8.1	Pitch Keypoint Detection	71
3.8.2	Homography Estimation and Coordinate Transformation	71
3.8.3	Caching and Robustness Measures	72
3.9	Tactical Output Generation	72
3.9.1	Bird’s-eye Visualisation	73
3.9.2	Heatmaps, Speed and Distance	73
3.9.3	Possession and Pass Detection	74
3.9.4	Offside Candidate Detection	74
3.10	Evaluation Methodology	75
3.10.1	Component-level Evaluation	75
3.10.2	End-to-end Pipeline Evaluation	76
3.10.3	Ablation Studies	76
3.10.4	Runtime Evaluation	77
3.10.5	Error Analysis	77
3.11	Implementation and Reproducibility	78
3.11.1	Software Environment	78
3.11.2	Configuration and Model Weights	79

3.11.3	Running the Pipeline	79
3.11.4	Reproducibility Limitations	80
3.12	Chapter Summary	80
4	Results	82
4.1	Experimental Setup	82
4.2	Findings	82
5	Evaluation	83
5.1	Analysis	83
5.2	Discussion	83
5.3	Limitations	83
6	Conclusion	84
6.1	Summary	84
6.2	Future Work	84
A	Additional Materials	91

Word Count: 12000 (TODO: update at the end)

List of Tables

1.1	Summary of evaluation metrics used in this project.	17
3.1	Data objects passed between pipeline stages. The <code>supervision.Detections</code> type is used throughout the left branch of the pipeline with fields progressively populated and, in the case of <code>class_id</code> , reinterpreted. . . .	58
3.2	Summary of the three training datasets. All datasets are hosted on Roboflow Universe and were downloaded via the Roboflow Python SDK; splits are those provided by the dataset version on Roboflow. . .	61

List of Figures

2.1	Simplified multi-layer perceptron. Inputs are passed through fully connected hidden layers using learned weights and non-linear activation functions before producing an output.	23
2.2	A common convolutional neural network architecture, consisting of repeated convolution, activation, and pooling stages followed by fully connected layers. Reproduced from O'Shea and Nash [35].	25
2.3	Intersection over Union (IoU) between a ground-truth bounding box B_g (blue, dashed) and a predicted bounding box B_p (orange). IoU is the ratio of the intersection area (gold) to the total union area of both boxes, computed as $\text{IoU} = B_g \cap B_p / B_g \cup B_p $	27
2.4	Illustration of Non-Maximum Suppression. Before NMS, multiple overlapping candidate boxes may be produced for the same object. After NMS, lower-confidence overlapping boxes are suppressed and the most confident detection is retained.	28
2.5	Generalised YOLO-style single-stage object detection pipeline. The input image is processed by a convolutional backbone to extract multi-scale feature maps, which are fused by the neck before the detection head predicts bounding boxes, class scores, and confidence scores. Non-Maximum Suppression (NMS) removes duplicate overlapping predictions to produce the final detection set $\{(b_i, c_i, s_i)\}$	31
2.6	ByteTrack two-stage association process. Detector output is partitioned by confidence threshold τ_{high} into a high-score set D_t^{high} and a low-score set D_t^{low} . Stage 1 matches active tracks against D_t^{high} using the Hungarian algorithm with IoU cost; unmatched tracks proceed to Stage 2, where they are compared against D_t^{low} . Tracks that remain unmatched after both stages are held in a buffer and deleted if unrecovered within a fixed number of frames.	35

2.7	Error propagation paths in a broadcast football analysis pipeline. Solid arrows show the main data flow between processing stages, while dashed arrows indicate how errors at each stage can affect downstream outputs.	48
3.1	End-to-end pipeline architecture. The broadcast frame enters at the top and is processed through two parallel branches—appearance-based detection and tracking on the left, pitch-geometry recovery on the right—which converge at the pitch-space projection stage. All higher-level tactical analytics operate in pitch coordinates. Section numbers on the left indicate where each stage is described in detail.	56
3.2	Effect of pixel displacement δ on IoU for square bounding boxes of varying size. A single-axis shift is applied to the predicted box while the ground-truth box remains fixed. At $\delta = 2$ pixels, a 16×16 box drops to $\text{IoU} = 0.78$ while a 64×64 box retains $\text{IoU} = 0.94$, illustrating why small-object detection is disproportionately sensitive to localisation error. The dashed line marks the standard mAP_{50} threshold. . .	64

List of Abbreviations

HOTA	Higher Order Tracking Accuracy
IDF1	Identification F1
IoU	Intersection over Union
MAE	Mean Absolute Error
mAP	mean Average Precision
MOTA	Multi-Object Tracking Accuracy

Abstract

TODO: my abstract (to be done at the end of the report)

Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

- i. The author of this report (including any appendices and/or schedules to this report) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii. Copies of this report, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the report, for example graphs and tables (“Reproductions”), which may be described in this report, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv. Further information on the conditions under which disclosure, publication and commercialisation of this report, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/>) and in The University’s policy on presentation of Theses

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Aphrodite Galata, for her invaluable guidance, support, and advice throughout this project.

I am also grateful to Kunj Dhola and Selim Sayed for their collaboration and contributions to the project.

Finally, I would like to thank my family and friends for their continued encouragement and support throughout my studies.

Chapter 1

Introduction

1.1 Motivation and Context

The use of data and analytics in professional football has grown substantially in recent years, with clubs and national federations increasingly incorporating analytical methods into performance analysis and decision-making [28]. In particular, positional and tracking data are valuable because they provide richer descriptions of player movement, team organisation, and tactical behaviour than traditional notational analysis alone [12, 45]. As a result, access to reliable spatial match data has become increasingly important.

Despite this, high-quality football data remain difficult to access outside elite settings. Advanced event and tracking data are often proprietary, expensive, or dependent on specialist collection systems, which limits their availability for public and reproducible work [25, 10, 7]. This motivates the investigation of alternative ways to generate structured football data from more widely available sources.

Broadcast video is an attractive candidate because it is widely available and already contains much of the spatial information needed for tactical analysis. However, extracting reliable structured data from broadcast footage is challenging, since it requires several interconnected tasks such as player detection, tracking, team assignment, and pitch registration under conditions including camera motion, occlusion, scale variation, and motion blur [26, 4, 44, 43].

Recent research has shown progress on many of these components and, increasingly, on parts of the broader workflow. Open benchmark efforts such as SoccerNet-Tracking and SoccerNet Game State Reconstruction demonstrate that broadcast football video can support tracking and game-state reconstruction in realistic settings [6,

41]. However, these systems typically produce intermediate outputs such as tracked player positions or minimap reconstructions rather than directly usable tactical outputs. What remains missing is a publicly reproducible end-to-end system that integrates all major stages from broadcast video to tactical output using modern computer vision methods.

1.2 Aims and Objectives

To address this gap, the project investigates how far a modular computer vision pipeline can extract reliable tactical and spatial information from broadcast football video without requiring specialised capture systems or manual team labels.

To achieve this aim, the project pursues the following objectives:

- develop and evaluate a detection approach for the main on-pitch entity classes—players, goalkeepers, referees, and the ball—from broadcast football footage;
- assess whether multi-object tracking can preserve sufficiently consistent identities over time to support downstream tactical analysis;
- test whether unsupervised clustering of visual embeddings can separate players by team without manual team labels;
- measure the geometric accuracy of pitch keypoint detection and homography estimation for projecting image-space positions into pitch-space;
- determine whether the resulting projected trajectories can support higher-level tactical outputs, including bird’s-eye visualisations, heatmaps, possession estimates, pass events, and offside-related analysis; and
- evaluate the pipeline as an end-to-end system, including how errors in detection, tracking, and homography estimation affect downstream analytical outputs.

The project does not aim to replace professional commercial tracking systems, but rather to assess how much tactical information a reproducible, open pipeline can extract from standard broadcast footage.

The project will be considered successful if: (i) the main object detector achieves $\text{mAP}_{50} \geq 0.80$ across entity classes; (ii) multi-object tracking reaches $\text{HOTA} \geq 0.45$ on a labelled test segment; (iii) unsupervised team assignment achieves $F_1 \geq 0.85$; and

(iv) the integrated pipeline produces coherent tactical outputs—including bird’s-eye projections, heatmaps, and offside analysis—that are qualitatively consistent with the underlying match footage. These thresholds reflect realistic performance expectations for broadcast football analysis informed by the literature [44, 6].

1.3 Evaluation Strategy

This project will be evaluated at both *component level* and *system level*. This is necessary because downstream outputs such as possession estimates, heatmaps, and offside decisions depend not only on the performance of individual modules, but also on how errors propagate across detection, tracking, and pitch registration stages [44, 43]. The evaluation therefore asks two questions: whether each component performs adequately on its own task, and whether the integrated pipeline produces coherent and tactically meaningful outputs as a complete system.

At component level, each module will be assessed using standard task-appropriate metrics from the computer vision literature. At system level, the pipeline will be judged by the coherence, plausibility, and usefulness of its final outputs on representative broadcast sequences. The chosen metrics reflect the main technical risks of the project: detection accuracy, temporal identity consistency, geometric projection quality, and the validity of downstream tactical inference. These metrics were chosen because each targets a distinct failure mode of the pipeline. Mean Average Precision (mAP) is the standard object detection metric in the computer vision literature and directly measures whether the detector can reliably localise and classify entities under broadcast conditions [44]. Higher Order Tracking Accuracy (HOTA) was selected over legacy Multi-Object Tracking (MOT) metrics such as Multi-Object Tracking Accuracy (MOTA) because it balances detection and association quality in a single score, making it better suited to evaluating identity consistency over time [29]. Reprojection error measures how accurately estimated homographies map known pitch points, which determines whether projected player positions are geometrically meaningful.

At system level, quantitative metrics alone cannot capture whether tactical outputs are useful, so qualitative assessment against the source footage is included following the approach used in recent game-state reconstruction benchmarks [41]. The numeric thresholds were set based on reported performance in comparable broadcast football settings: for example, Theiner et al. [44] report mAP₅₀ values in the range 0.80–0.90 for player detection, and SoccerNet-Tracking baselines report HOTA scores

in the 0.40–0.55 range [6], making the chosen targets realistic but non-trivial. Detailed annotation procedures and experimental methodology are presented in Chapter 5.

Table 1.1: Summary of evaluation metrics used in this project.

Component / Output	Primary metrics	Purpose
Object detection	mAP _{50–95} , mAP ₅₀ , precision, recall	Measure localisation and classification performance for players, goalkeepers, referees, and the ball.
Ball detection	mAP ₅₀ , precision, recall, false positive rate	Assess robustness under small object size, motion blur, and occlusion.
Pitch keypoints and homography	mAP _{50–95} for keypoints; reprojection error for homography	Measure field registration quality and projection accuracy in pitch-space.
Multi-object tracking	HOTA, IDF1, MOTA, identity switches	Evaluate temporal consistency and identity preservation [29, 9].
Team assignment	Precision, recall, F_1 score	Measure accuracy of unsupervised team separation.
Tactical outputs	Precision, recall, and F_1 for event detection; MAE for possession; qualitative assessment for heatmaps and bird’s-eye views	Assess whether projected trajectories support useful downstream analysis.
End-to-end pipeline	Integrated case study and failure analysis	Evaluate output coherence and identify how upstream errors affect downstream reasoning.

Component-level evaluation shows whether each stage meets its technical objective, while system-level evaluation tests whether the combined pipeline remains analytically useful once these stages are linked. The final judgement of success will therefore be based on both quantitative component metrics and qualitative assessment of whether the integrated outputs remain consistent with the source footage and useful for tactical interpretation.

1.4 Report Structure

The remainder of this report is organised to guide the reader from the background knowledge needed to understand the system, through its design and outputs, to a critical assessment of its performance. Chapter 2 (*Background and Theory*) reviews the relevant literature and technical foundations, establishing the context from which the pipeline design decisions follow. Chapter 3 (*Methodology and Implementation*) describes the architecture and implementation of each pipeline stage, building directly on

the methods and models introduced in the background. Chapter 4 (*Results*) presents the outputs produced by the system in practice. Chapter 5 (*Evaluation*) then assesses the project at both component and system level, using the metrics and strategy outlined in Section 1.3, including failure analysis and the effect of error propagation on downstream outputs. Finally, Chapter 6 (*Conclusion*) summarises the main achievements, reflects on limitations, and outlines directions for future work.

Chapter 2

Background and Theory

2.1 Broadcast Football Video Analysis

Extracting structured information from broadcast football footage poses several coupled technical challenges that distinguish it from analysis based on dedicated tracking infrastructure [44, 26]. Unlike specialised systems that rely on multi-camera installations or stadium-side sensors, a broadcast pipeline must recover all player, ball, and pitch information from a single compressed, moving-camera video stream [12].

Several properties of broadcast footage make this challenging. First, the camera continuously pans, tilts, and zooms to follow play, so the background cannot be treated as a fixed spatial reference and inter-frame motion cannot be attributed to scene objects alone [44]. Second, the ball is extremely small relative to the frame and is frequently blurred, occluded, or temporarily absent, making reliable detection substantially harder than for players [14]. Third, players on the same team share near-identical visual appearance due to uniform colour and similar build, so tracking and team identification must rely on subtle spatial and visual cues [44]. Fourth, image-space measurements are subject to perspective distortion, so quantities such as distance, speed, and tactical positioning are meaningful only after registration to a canonical pitch model [43].

A widely adopted response to this complexity is to decompose the problem into a sequence of specialised components [44, 26, 12]: object detection and ball detection provide the primary observations, multi-object tracking maintains temporal identity, team assignment infers player affiliation, pitch registration maps image coordinates into a canonical field representation, and higher-level outputs such as heatmaps, possession estimates, and offside reasoning are then expressed in pitch space rather than image coordinates.

2.2 Related Work

The previous section established that broadcast football analysis requires several connected computer vision tasks. This section reviews existing commercial systems, public benchmarks, and academic pipelines that address this problem. The aim is to identify where current work is strong, and where there remains space for an open pipeline that produces tactical outputs from broadcast video.

2.2.1 Commercial Tracking Systems

In professional football, positional data are commonly collected using commercial optical or sensor-based tracking systems. TRACAB, for example, uses multi-camera optical tracking to estimate player and ball positions, with Linke et al. [25] reporting sub-metre positional accuracy under validated match conditions. Other providers, such as Hudl StatsBomb and Second Spectrum, offer event data, tracking data, and tactical analysis tools for professional clubs and broadcasters [7].

These systems demonstrate the practical value of tracking data in football analysis. However, they are less suitable as a foundation for open research because the underlying algorithms, data collection procedures, and raw tracking data are often proprietary. This makes it difficult to reproduce results, compare methods fairly, or apply the same approach to matches where commercial tracking data are unavailable [10, 7]. This limitation motivates research into methods that can recover useful spatial and tactical information directly from broadcast footage.

2.2.2 Academic Benchmarks and Integrated Pipelines

The SoccerNet ecosystem has become one of the main open benchmark efforts for football video understanding. SoccerNet-v2 introduced large-scale annotations for tasks such as action spotting, camera shot segmentation, and replay detection across broadcast matches [8]. SoccerNet-Tracking extended this work to multi-object tracking by providing player bounding-box annotations and standard evaluation protocols for tracking in broadcast footage [6]. More recently, SoccerNet Game State Reconstruction framed the problem as reconstructing the game state on a top-down minimap, combining detection, tracking, identification, and pitch registration in a single benchmark task [41].

Several academic systems have also addressed important parts of the broadcast

analysis pipeline. Theiner et al. [44] proposed a pipeline for extracting positional data from broadcast football video by combining detection, tracking, and pitch registration. Their work shows that broadcast footage can be used to estimate player positions, but the focus is mainly on positional accuracy rather than downstream tactical reasoning. Magera et al. [31] proposed BroadTrack, which improves broadcast camera calibration and registration accuracy, but concentrates on the registration stage rather than the full analysis pipeline. Maglo et al. [32] presented an end-to-end approach for localising players from a single moving broadcast view, combining field registration with self-supervised re-identification. Earlier work by Beetz et al. [4] also demonstrated the potential of automated sports game analysis, although it predates modern deep-learning based detection and tracking methods.

A common pattern in this literature is that many systems stop at intermediate representations, such as bounding boxes, tracks, pitch registration, or minimap reconstruction. These outputs are important, but they are not the same as tactical analysis. A reconstruction can be visually plausible while still containing localisation, identity, or ball-detection errors that affect higher-level outputs such as possession, pass detection, distance estimates, or offside decisions. Therefore, evaluating only the intermediate representation does not fully answer whether the system is useful for tactical interpretation.

2.2.3 Positioning of This Project

The literature reviewed above leaves a clear gap between closed commercial systems and open academic reconstruction benchmarks. Commercial providers demonstrate that tactical outputs can be valuable in practice, but their methods and data are not generally reproducible. Academic work, by contrast, provides open benchmarks and methods for detection, tracking, and pitch registration, but often evaluates success at the level of reconstruction rather than tactical interpretation.

This project addresses that gap by extending a broadcast-video pipeline beyond intermediate outputs towards pitch-space tactical analysis. Rather than treating detection, tracking, and registration as final outputs, they are used as inputs to downstream statistics and tactical reasoning. The implications of this design, including its benefits and limitations, are evaluated in Chapter 5.

2.3 Machine Learning and Computer Vision Foundations

The pipeline developed in this project relies on several machine learning and computer vision ideas. This section introduces the required background at a high level before the later sections discuss the project-specific methods in more detail. The aim is not to give a complete survey of machine learning, but to explain the concepts needed to understand why trained visual models are used for detection, team identification, and pitch analysis in broadcast football video.

2.3.1 Supervised Learning for Visual Recognition

Supervised learning is a machine learning setting in which a model learns from labelled examples. A training dataset consists of input-output pairs

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N,$$

where x_i is an input, such as an image or image crop, and y_i is the corresponding label or annotation. In visual recognition tasks, the label might be an object class, a bounding box, a set of keypoints, or a combination of these. The model is represented by a function f_θ , where θ denotes the learnable parameters. Training aims to find parameters that minimise a loss function over the labelled examples:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_\theta(x_i), y_i).$$

The loss function measures the difference between the model prediction and the ground-truth label, and optimisation adjusts the model parameters to reduce this error [13].

This idea is central to the supervised components of the project. The object detectors learn to predict bounding boxes and classes for players, goalkeepers, referees, and the ball. Similarly, the pitch keypoint detector learns to localise meaningful landmarks on the football pitch. In each case, the model is not given explicit hand-written rules for recognising the object; instead, it learns visual patterns from annotated examples.

2.3.2 Neural Networks and Multi-Layer Perceptrons

A neural network is a parametric model composed of layers of simple computational units. Each unit, or neuron, receives inputs, combines them using learned weights and biases, and then applies a non-linear activation function. In a multi-layer perceptron (MLP), neurons are arranged into an input layer, one or more hidden layers, and an output layer, as illustrated in Figure 2.1.

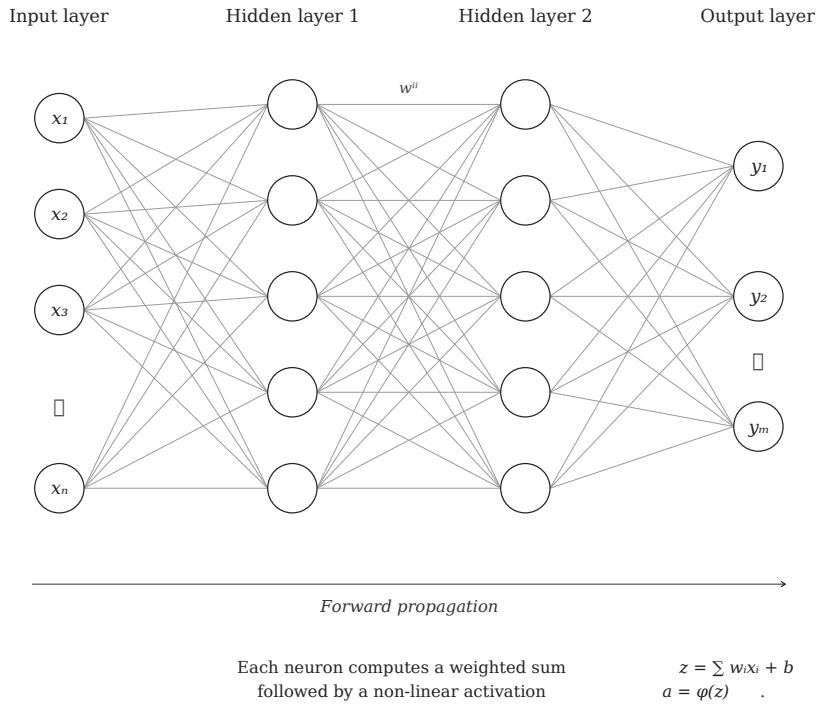


Figure 2.1: Simplified multi-layer perceptron. Inputs are passed through fully connected hidden layers using learned weights and non-linear activation functions before producing an output.

For layer l , the output can be written as

$$\mathbf{h}^{(l)} = \sigma \left(W^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right),$$

where $\mathbf{h}^{(l-1)}$ is the input to the layer, $W^{(l)}$ is a matrix of learned weights, $\mathbf{b}^{(l)}$ is a learned bias vector, and σ is a non-linear activation function. During training, these weights and biases are adjusted so that the network's predictions better match the labelled training data [13].

The non-linear activation function is essential. Without it, stacking several linear

layers would still be equivalent to a single linear transformation, limiting the model's ability to represent complex relationships. With non-linear activations, neural networks can learn more flexible mappings, making them useful for tasks such as classification, localisation, and feature extraction.

However, a standard MLP treats its input as a flat vector. This is inefficient for images because flattening an image ignores the spatial structure of neighbouring pixels and leads to a large number of parameters. Convolutional neural networks address this limitation by using operations designed specifically for image data.

2.3.3 Convolutional Neural Networks

Convolutional neural networks (CNNs) are neural networks designed for grid-structured data such as images. A standard multi-layer perceptron treats an image as a flat vector, which ignores the spatial relationship between neighbouring pixels. CNNs address this by applying small learned filters across local regions of the image. This allows the same visual pattern, such as an edge or texture, to be detected regardless of where it appears in the frame.

A simplified two-dimensional convolution can be written as

$$y_{i,j,k} = \sigma \left(\sum_{u,v,c} W_{u,v,c,k} x_{i+u,j+v,c} + b_k \right),$$

where x is the input feature map, W is a learned convolutional filter, b_k is a bias term, σ is a non-linear activation function, and $y_{i,j,k}$ is the output activation at spatial location (i, j) for channel k . The indices u and v range over the spatial extent of the filter, while c ranges over the input channels. Because the same filter weights are reused across the image, CNNs require fewer parameters than fully connected networks and are better suited to visual data.

Figure 2.2 shows a common CNN structure. Convolutional layers produce feature maps by applying learned filters to the input. Activation functions such as ReLU introduce non-linearity, allowing the network to model more complex visual relationships. Pooling layers reduce the spatial resolution of feature maps, which lowers computation and can make the representation less sensitive to small translations in the input.

CNNs are well suited to visual recognition because they learn hierarchical representations. Early layers often capture local visual patterns such as edges, colour contrasts, and textures, while deeper layers combine these into more semantic features

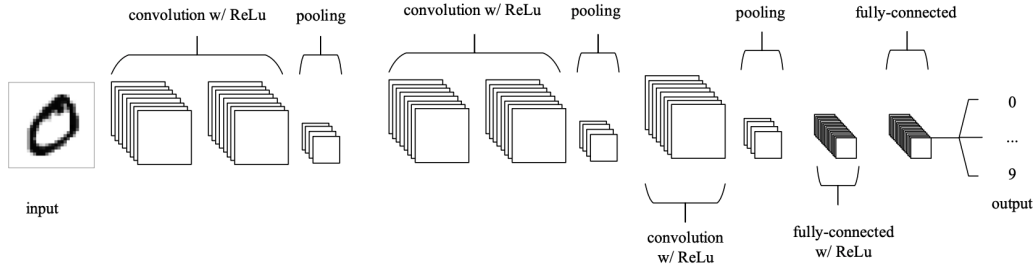


Figure 2.2: A common convolutional neural network architecture, consisting of repeated convolution, activation, and pooling stages followed by fully connected layers. Reproduced from O’Shea and Nash [35].

such as body parts, objects, or pitch markings. This idea was present in early convolutional systems such as LeNet [22], and deep CNNs became dominant in modern computer vision after their success on large-scale image classification benchmarks [20].

In this project, CNN-based models provide the visual backbone for detection and keypoint localisation. A detector such as YOLO does not rely on hand-written rules applied directly to raw pixels. Instead, convolutional layers extract feature maps from broadcast frames, and prediction heads use these features to estimate object classes, bounding boxes, or pitch keypoints. The architecture and detection-specific details of YOLO are discussed in Section 2.4.

2.3.4 Transfer Learning and Fine-Tuning

Training deep visual models from scratch usually requires large datasets and substantial computation. In many applied computer vision projects, a more practical approach is transfer learning. A model is first trained on a large general dataset and then adapted to a more specific target task. The lower layers of a CNN often learn general visual features, such as edges and textures, that are useful across many domains, while later layers become more task-specific [46].

Fine-tuning is the process of continuing training from a pre-trained model using task-specific data. This is useful for broadcast football analysis because the available labelled football data are much smaller than general image datasets, but the task still benefits from strong visual features learned from large-scale pre-training. In this project, fine-tuning allows detection and keypoint models to be specialised for football footage without requiring all visual representations to be learned from the beginning.

Transfer learning also helps with robustness. Broadcast video contains variation

in camera angle, scale, lighting, kit colour, motion blur, and occlusion. A model initialised from general visual features can often adapt more effectively to these conditions than a model trained only from a small football-specific dataset.

2.4 Object Detection in Broadcast Football

Object detection is the task of locating and classifying objects within an image. In broadcast football analysis, the main objects of interest are players, goalkeepers, referees, and the ball. For each detected object, the model must predict both where the object is in the frame and what class it belongs to. This stage is foundational because errors at this point propagate into later stages of the pipeline: missed players affect tracking, inaccurate boxes affect pitch projection, and missed ball detections affect possession and pass reasoning.

Formally, a detector maps an input image I to a set of predictions

$$D = \{(b_i, c_i, s_i)\}_{i=1}^N, \quad (2.1)$$

where b_i denotes a bounding box, c_i is the predicted class label, and $s_i \in [0, 1]$ is a confidence score. The confidence score reflects the detector’s certainty that an object of class c_i is present at location b_i .

2.4.1 Bounding Boxes, IoU, and Non-Maximum Suppression

A bounding box is a rectangular region that encloses an object in the image. It may be represented by its corner coordinates or by its centre position, width, and height. To evaluate how well a predicted box matches a ground-truth annotation, object detection systems commonly use Intersection over Union (IoU). For a predicted bounding box B_p and a ground-truth bounding box B_g , IoU is defined as

$$\text{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}, \quad (2.2)$$

where $B_p \cap B_g$ is the overlapping area and $B_p \cup B_g$ is the total area covered by both boxes. IoU takes values between 0 and 1. A value close to 1 indicates strong overlap, while a value close to 0 indicates poor localisation. Figure 2.3 illustrates this relationship.

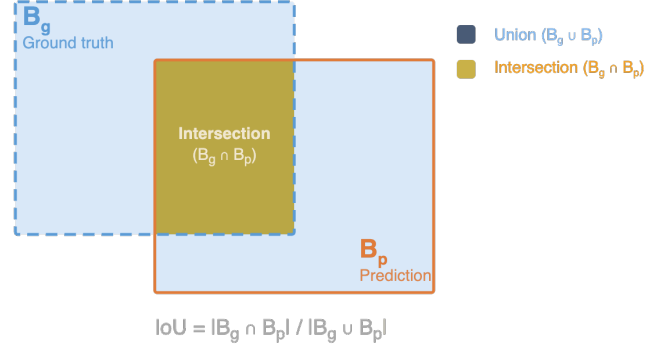


Figure 2.3: Intersection over Union (IoU) between a ground-truth bounding box B_g (blue, dashed) and a predicted bounding box B_p (orange). IoU is the ratio of the intersection area (gold) to the total union area of both boxes, computed as $\text{IoU} = |B_g \cap B_p| / |B_g \cup B_p|$.

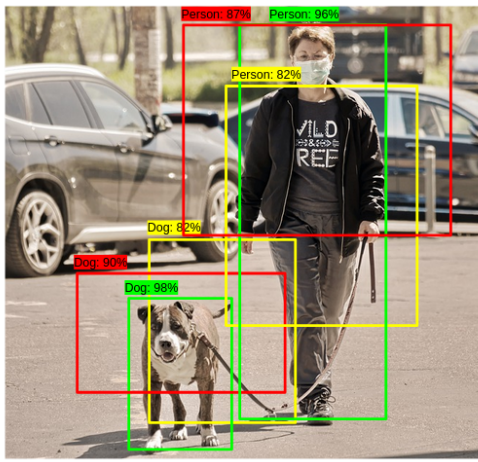
In evaluation, a prediction is usually counted as a true positive only if its IoU with an unmatched ground-truth box exceeds a chosen threshold and the predicted class is correct [24]. For example, an IoU threshold of 0.50 means that the predicted and ground-truth boxes must overlap by at least half of their combined area.

Object detectors often produce several overlapping predictions for the same object. Non-Maximum Suppression (NMS) is therefore applied after prediction to remove duplicate boxes. The detections are first sorted by confidence score. The highest-confidence detection is kept, and lower-confidence detections with high IoU overlap with it are suppressed. This process is repeated until no remaining detections overlap too strongly with a selected box [37]. Figure 2.4 illustrates this process.

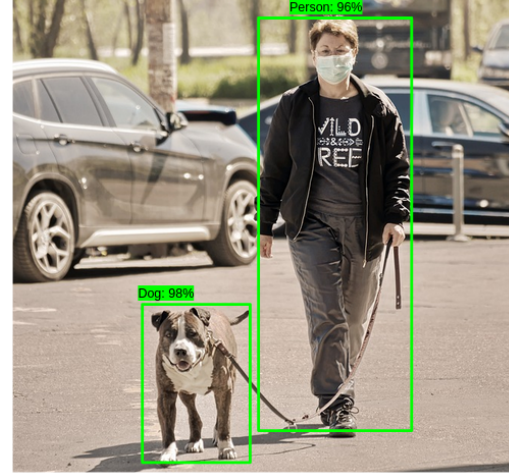
NMS is important in football footage because players can appear close together, especially during tackles, set pieces, and crowded midfield situations. Without NMS, the same player could be counted multiple times, which would create errors in tracking and downstream analysis.

2.4.2 Precision, Recall, and Mean Average Precision

After detections have been matched to ground-truth annotations using an IoU threshold, performance can be measured using precision and recall. Precision measures how many predicted detections are correct, while recall measures how many ground-truth



(a) Before NMS



(b) After NMS

Figure 2.4: Illustration of Non-Maximum Suppression. Before NMS, multiple overlapping candidate boxes may be produced for the same object. After NMS, lower-confidence overlapping boxes are suppressed and the most confident detection is retained.

objects are successfully detected:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (2.3)$$

where TP, FP, and FN denote true positives, false positives, and false negatives respectively. In the context of this project, a false positive might be an incorrectly detected player or ball, while a false negative might be a player or ball that is missed by the detector.

Object detectors produce confidence scores, so changing the confidence threshold changes the balance between precision and recall. A high threshold usually improves precision by removing uncertain detections, but may reduce recall because more true objects are missed. A low threshold usually improves recall, but may introduce more false positives. This trade-off is summarised by the precision–recall curve.

Average Precision (AP) measures the area under the precision–recall curve for one class, and Mean Average Precision (mAP) is the mean AP across all object classes [24]. In multi-class football detection, this is important because different classes have different levels of difficulty. Players and referees are relatively large, while the ball is much smaller and harder to localise. Reporting only a single overall score can therefore hide poor performance on difficult classes, so per-class AP values are also useful.

Two common detection metrics are mAP_{50} and mAP_{50-95} . The first evaluates detections using an IoU threshold of 0.50. The second averages mAP over IoU thresholds from 0.50 to 0.95 in steps of 0.05, making it a stricter measure of localisation quality. This distinction is important for broadcast football analysis because a loosely localised player box may still be acceptable for rough visualisation, but accurate boxes are needed for tracking, pitch projection, and downstream tactical analysis.

2.4.3 The YOLO Architecture

Before discussing YOLO specifically, it is useful to distinguish between two broad families of object detectors. Two-stage detectors, such as Faster R-CNN [40], first generate candidate object regions and then classify and refine those regions in a second stage. This can provide strong localisation accuracy, but the additional proposal stage increases computational cost. Single-stage detectors, such as SSD [27] and YOLO [37], predict bounding boxes and class scores directly in one forward pass. This makes them well suited to video-rate applications such as broadcast football analysis, where detections must be produced for many consecutive frames.

YOLO, short for *You Only Look Once*, is a family of single-stage object detectors designed to perform object localisation and classification efficiently. The original YOLO formulation treated object detection as a single regression problem: an image was divided into a coarse grid, and each grid cell predicted bounding boxes, confidence scores, and class probabilities [37]. This design allowed YOLO to run in real time, although early versions struggled with small objects and crowded scenes.

Later YOLO versions improved this basic idea. YOLOv2 introduced anchor boxes and multi-scale training, improving recall for objects of different sizes [38]. YOLOv3 predicted detections at multiple spatial resolutions, which improved performance on both small and large objects [39]. More recent YOLO variants, including YOLOv8, use anchor-free detection heads, where bounding-box coordinates are regressed directly from feature-map locations rather than from pre-defined anchor boxes [17]. This reduces the need for hand-designed anchor priors and simplifies adaptation to new domains.

At a high level, a modern YOLO-style detector works as follows:

1. The input image is passed through a convolutional backbone, which extracts visual feature maps at multiple spatial resolutions.
2. A neck, often based on feature-pyramid ideas, combines low-level spatial detail

with high-level semantic information across scales [23].

3. A detection head predicts bounding-box coordinates, class scores, and confidence scores from the fused feature maps.
4. Low-confidence predictions are removed using a confidence threshold.
5. Non-Maximum Suppression is applied to remove duplicate overlapping boxes and produce the final detection set.

Figure 2.5 shows this architecture at an abstract level. This structure is important for broadcast football footage because the objects of interest appear at very different scales. A player close to the camera may occupy a large part of the frame, while a player on the far side of the pitch or the ball may occupy only a small number of pixels. Multi-scale feature extraction helps the detector combine fine spatial detail with broader contextual information, improving detection across these scale changes.

In this project, YOLO-based detectors provide the frame-level observations that are passed to the tracking, team identification, pitch registration, and tactical analysis stages.

2.5 Multi-Object Tracking

Object detection produces independent bounding boxes in each frame, but it does not by itself say whether a player detected in one frame is the same player detected in the next. Multi-object tracking (MOT) addresses this problem by linking detections over time and assigning a persistent identity to each object [9]. In this project, tracking is necessary because downstream outputs such as player heatmaps, distance estimates, speed estimates, team assignment, possession reasoning, and offside analysis depend on following the same player across multiple frames.

Formally, let

$$D_t = \{d_t^1, d_t^2, \dots, d_t^{n_t}\} \quad (2.4)$$

denote the set of detections at frame t , where each detection contains a bounding box, class label, and confidence score. Let

$$\mathcal{T}_{t-1} = \{\tau_{t-1}^1, \tau_{t-1}^2, \dots, \tau_{t-1}^{m_{t-1}}\} \quad (2.5)$$

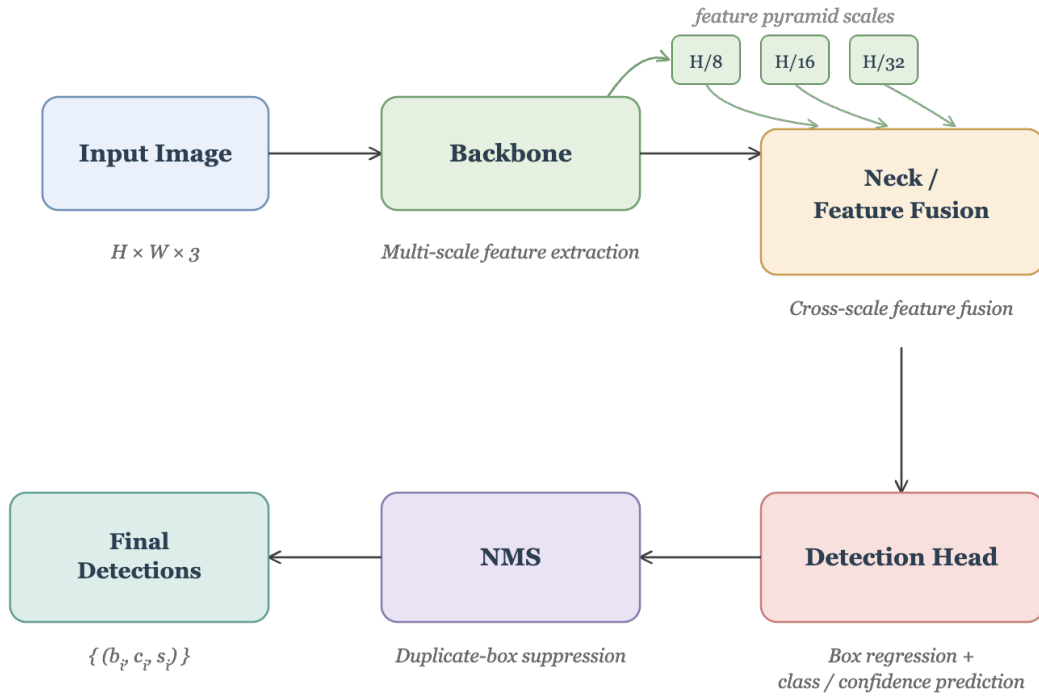


Figure 2.5: Generalised YOLO-style single-stage object detection pipeline. The input image is processed by a convolutional backbone to extract multi-scale feature maps, which are fused by the neck before the detection head predicts bounding boxes, class scores, and confidence scores. Non-Maximum Suppression (NMS) removes duplicate overlapping predictions to produce the final detection set $\{(b_i, c_i, s_i)\}$.

denote the set of active tracks from the previous frame. The tracking problem is to assign detections in D_t to existing tracks in $\mathcal{T}_{t-1}$, while creating new tracks for unmatched detections and handling tracks that temporarily receive no matching detection.

2.5.1 Tracking-by-Detection

A common approach to MOT is *tracking-by-detection* [5]. In this paradigm, an object detector is first applied independently to each frame, and a separate tracking algorithm then associates the resulting detections across time. This separates the visual recognition problem from the temporal association problem: the detector answers “what objects are visible in this frame?”, while the tracker answers “which detections belong to the same object over time?”.

Tracking-by-detection methods usually follow three main steps:

1. **Detection:** an object detector produces candidate bounding boxes and confidence scores for the current frame.
2. **Prediction:** the tracker predicts where each existing track is expected to appear in the current frame, usually using a motion model.
3. **Association:** predicted track locations are matched to current detections using a similarity or cost measure, such as IoU.

This framework is widely used because it is modular and efficient. Improvements in object detection can be incorporated without redesigning the whole tracker, and the association stage can be performed online, frame by frame. This is important for video applications because the tracker does not need to wait for the full video sequence before producing trajectories.

However, tracking-by-detection is also sensitive to the quality of the detector. Missed detections can cause tracks to fragment, false positives can create spurious tracks, and inaccurate bounding boxes can lead to incorrect associations. These failure cases are particularly important in crowded scenes, where several visually similar objects may be close together. The next subsection introduces the two main mechanisms used to reduce these problems: motion prediction and data association.

2.5.2 Motion Prediction and Data Association

Tracking-by-detection requires a way to decide which detection in the current frame corresponds to which existing track. This is usually handled using two linked ideas:

motion prediction and data association. Motion prediction estimates where each tracked object is likely to appear in the next frame, while data association matches these predictions to the detections produced by the object detector.

A common motion model is the Kalman filter, which estimates the state of a system over time from noisy observations [18]. In object tracking, the state vector typically contains the bounding-box position and size, together with their velocities. A simplified state for a tracked bounding box can be written as

$$\mathbf{x}_t = [x_t, y_t, w_t, h_t, \dot{x}_t, \dot{y}_t, \dot{w}_t, \dot{h}_t]^\top,$$

where (x_t, y_t) denotes the box centre, w_t and h_t denote width and height, and the dotted terms denote their estimated rates of change.

The Kalman filter alternates between a prediction step and an update step. In the prediction step, the previous state is propagated forward using a motion model:

$$\hat{\mathbf{x}}_t^- = F\mathbf{x}_{t-1}, \quad P_t^- = FP_{t-1}F^\top + Q, \quad (2.6)$$

where F is the state-transition matrix, P_t^- is the predicted covariance, and Q represents process noise. The predicted state $\hat{\mathbf{x}}_t^-$ gives the expected location of the track before seeing the detector output for frame t .

When a detection is matched to the track, the update step corrects the prediction using the measurement $\mathbf{z}_t$:

$$\mathbf{x}_t = \hat{\mathbf{x}}_t^- + K_t (\mathbf{z}_t - H\hat{\mathbf{x}}_t^-), \quad (2.7)$$

where H maps the state into measurement space and K_t is the Kalman gain. The term $\mathbf{z}_t - H\hat{\mathbf{x}}_t^-$ is the difference between the observed detection and the predicted measurement. The Kalman gain controls how strongly the tracker should trust the new detection compared with the motion prediction.

After prediction, the tracker must assign current detections to existing tracks. This is a bipartite matching problem: each track should be matched to at most one detection, and each detection should be matched to at most one track. A cost matrix C is constructed, where each entry C_{ij} measures how incompatible track i is with detection j . A common choice is

$$C_{ij} = 1 - \text{IoU}(\hat{B}_i, B_j), \quad (2.8)$$

where $\hat{B}_i$ is the predicted bounding box for track i , and B_j is the bounding box of

detection j . Lower cost means a better match.

The Hungarian algorithm solves this assignment problem by finding the minimum-cost one-to-one matching between tracks and detections [21]. Detections that are not matched to any existing track can initialise new tracks, while tracks that are not matched may be kept temporarily in a buffer or eventually deleted. This combination of Kalman filtering and Hungarian assignment forms the basis of several online tracking methods, including SORT [5], ByteTrack [48], and BoT-SORT [2].

2.5.3 ByteTrack

ByteTrack is a multi-object tracking algorithm designed for tracking-by-detection pipelines [48]. Its main idea is simple but effective: instead of using only high-confidence detections and discarding the rest, ByteTrack also uses low-confidence detections during a second association stage. This is useful because a low detector confidence does not always mean that the object is false. The confidence may drop because of occlusion, motion blur, truncation, or poor image quality, while the object itself may still be present.

For each frame, ByteTrack separates detections into two sets according to a confidence threshold:

$$D_t^{\text{high}} = \{d \in D_t : s_d \geq \tau_{\text{high}}\}, \quad (2.9)$$

and

$$D_t^{\text{low}} = \{d \in D_t : \tau_{\text{low}} \leq s_d < \tau_{\text{high}}\}, \quad (2.10)$$

where s_d is the confidence score of detection d , τ_{high} is the high-confidence threshold, and τ_{low} is a lower threshold used to remove very unreliable detections.

The association process then proceeds in two stages:

1. Existing tracks are first matched with high-confidence detections D_t^{high} , usually using IoU-based costs and the Hungarian algorithm.
2. Tracks that remain unmatched are then matched with low-confidence detections D_t^{low} . This can recover objects whose detector confidence has temporarily fallen.
3. Unmatched high-confidence detections initialise new tracks if they satisfy the required threshold.
4. Tracks that remain unmatched after both association stages are not deleted immediately. They are kept for a short buffer period and removed only if they are

not recovered.

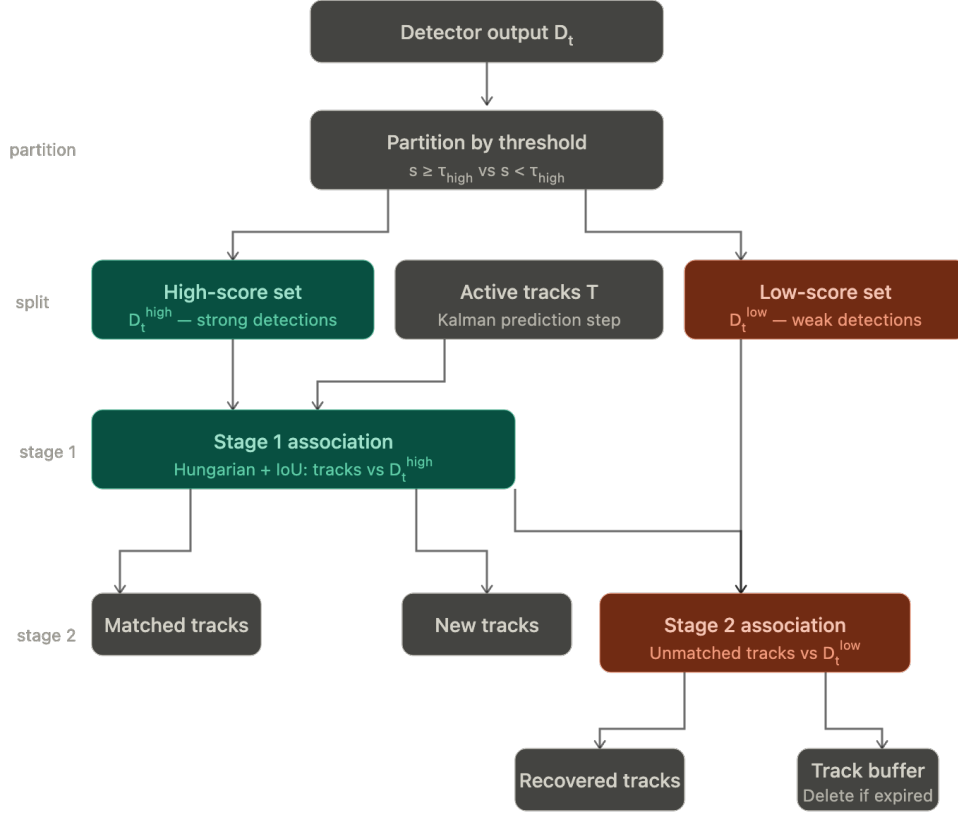


Figure 2.6: ByteTrack two-stage association process. Detector output is partitioned by confidence threshold τ_{high} into a high-score set D_t^{high} and a low-score set D_t^{low} . Stage 1 matches active tracks against D_t^{high} using the Hungarian algorithm with IoU cost; unmatched tracks proceed to Stage 2, where they are compared against D_t^{low} . Tracks that remain unmatched after both stages are held in a buffer and deleted if unrecovered within a fixed number of frames.

As illustrated in Figure 2.6, this use of low-confidence detections helps preserve track continuity during short occlusions, motion blur, or weak detections, while still relying on high-confidence detections for the most reliable associations.

2.5.4 BoT-SORT as an Alternative Tracker

BoT-SORT extends the tracking-by-detection framework by adding two mechanisms that are useful in challenging video: camera-motion compensation and appearance-based re-identification [2]. Like ByteTrack, it uses detections as input and associates them with existing tracks over time. However, it attempts to improve association when motion prediction based only on bounding-box overlap is unreliable.

Camera-motion compensation is useful when the camera itself moves between frames. In a static-camera setting, changes in object position are mainly caused by object motion. In broadcast video, however, camera panning, tilting, and zooming can cause all objects in the frame to shift together. BoT-SORT compensates for this global image motion before matching detections to tracks, making the predicted track positions more consistent with the current frame.

BoT-SORT can also incorporate appearance-based re-identification features. Instead of relying only on spatial overlap, the tracker can compare visual features extracted from object crops. This helps distinguish objects when their bounding boxes are close together or when short occlusions occur. The trade-off is that appearance feature extraction and camera-motion compensation add computational cost and implementation complexity.

2.5.5 Tracking Evaluation: HOTA

Tracking quality is assessed using metrics that separately capture localisation accuracy and identity consistency. Multi-Object Tracking Accuracy (MOTA) and Identification F1 (IDF1) are established baselines [9], but both conflate detection and association performance in ways that can obscure the source of errors.

Higher Order Tracking Accuracy (HOTA) was introduced to address this by decomposing overall performance into a detection score and an association score [29]:

$$\text{HOTA}_\alpha = \sqrt{\text{DetA}_\alpha \cdot \text{AssA}_\alpha}, \quad (2.11)$$

where DetA_α measures how well predicted boxes overlap ground-truth boxes at localisation threshold α , and AssA_α measures how consistently predicted identities match ground-truth identities across time. The geometric mean ensures that neither component can dominate: a tracker that localises well but fragments identities, or associates well but misses many objects, will score poorly overall. The final HOTA score is averaged over $\alpha \in [0.05, 0.95]$.

2.6 Team Identification from Visual Features

Object detection and tracking can identify where players are and how they move, but they do not directly determine which team each player belongs to. In broadcast football footage, team identity is usually not available as an explicit detector output, and manually labelling every player for each match would reduce the usefulness of an automated pipeline. Since players from the same team usually share a visually similar kit, team identification can be treated as an unsupervised grouping problem based on player appearance [16].

2.6.1 Team Identification as Clustering

In this setting, each detected player is first represented by a feature vector that describes their visual appearance. The aim is that players wearing the same kit produce similar feature vectors, while players from opposing teams produce different feature vectors. A clustering algorithm can then group these features without requiring match-specific team labels. Clustering is a standard unsupervised learning approach in which data points are partitioned into groups according to similarity [30].

Formally, let I_i denote a crop of player i , and let f be a feature extractor that maps the crop to a vector representation:

$$\mathbf{z}_i = f(I_i), \quad \mathbf{z}_i \in \mathbb{R}^d. \quad (2.12)$$

The team-identification problem is then to partition the set of feature vectors $\{\mathbf{z}_i\}_{i=1}^N$ into groups corresponding to the teams on the pitch. For outfield players, this is usually a two-cluster problem because the two teams normally wear visually distinct kits.

2.6.2 Visual Representations for Team Identification

The quality of team identification depends strongly on how player appearance is represented before clustering. The simplest representation is based on colour. For example, colour histograms can be extracted from player crops in colour spaces such as RGB, HSV, or Lab, and players can then be grouped according to the similarity of these histograms [16]. This approach is fast and directly related to the main visual cue used by humans: the colour of the team kit.

However, colour-only features have limitations. They can be affected by lighting changes, shadows, compression artefacts, and background pixels included in the player

crop. They also ignore spatial structure, so two crops with similar colour distributions but different visual patterns may appear similar to the clustering algorithm. This can be problematic when the two teams have partially similar colours, or when referees and goalkeepers wear colours that overlap with one of the teams.

A richer alternative is to use deep visual embeddings. Instead of describing a crop only by its colour distribution, a pretrained visual encoder maps the crop to a high-dimensional vector that can capture colour, texture, shape, and pattern information. Modern models such as CLIP, SigLIP, and DINOv2 learn transferable visual representations from large-scale pre-training, making them useful feature extractors even when no match-specific team labels are available [36, 47, 34]. For team identification, these embeddings can provide a more robust representation of kit appearance than colour histograms alone.

2.6.3 Dimensionality Reduction and K-Means

Deep visual embeddings are often high-dimensional, which can make direct clustering less stable and harder to interpret. In high-dimensional spaces, distances between points can become less informative because many points appear similarly far apart, a problem often described as the curse of dimensionality [1]. Dimensionality reduction can therefore be used to map embeddings into a lower-dimensional space before clustering.

Let $\mathbf{z}_i \in \mathbb{R}^d$ be the embedding of player crop i . A dimensionality-reduction method defines a mapping

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}, \quad d' \ll d, \quad (2.13)$$

producing a reduced feature vector

$$\mathbf{y}_i = \phi(\mathbf{z}_i). \quad (2.14)$$

Principal Component Analysis (PCA) is a common linear method that preserves directions of maximum variance, while UMAP is a non-linear method designed to preserve local neighbourhood structure [33]. For team identification, preserving local similarity can be useful because players wearing the same kit should occupy nearby regions of feature space.

After dimensionality reduction, k -means clustering [30] can be used to partition the feature vectors into teams. Given reduced features $\{\mathbf{y}_i\}_{i=1}^N$, k -means finds k cluster

centroids by minimising the within-cluster sum of squared distances:

$$\min_{\{\mu_j\}_{j=1}^k} \sum_{i=1}^N \min_{1 \leq j \leq k} \|\mathbf{y}_i - \mu_j\|_2^2, \quad (2.15)$$

where μ_j is the centroid of cluster j . For the two outfield teams, $k = 2$ is the natural choice. The resulting clusters provide team assignments up to an arbitrary label swap, since the algorithm can separate the two groups but does not know which cluster corresponds to which named team.

2.6.4 Temporal Stability and Special Cases

Clustering individual player crops can produce noisy team labels because each crop is affected by pose, blur, occlusion, lighting, and background pixels. However, a player's team identity should remain constant over a track. This means that temporal information can be used to make team assignment more stable.

A simple approach is to aggregate predicted team labels over multiple frames rather than relying on a single crop. For example, if a tracked player is assigned to one cluster in most frames, the majority label can be used as the final team label for that track. This reduces short-term jitter caused by poor-quality crops or temporary visual ambiguity. More generally, temporal smoothing uses neighbouring frames to make predictions more consistent over time.

Goalkeepers are a special case because they usually wear a kit that differs from both outfield teams. As a result, a clustering method designed to separate the two outfield teams may assign goalkeepers inconsistently or place them in the wrong cluster. Referees present a similar issue because their appearance is also distinct from the two teams. For these cases, visual clustering alone may be insufficient, and additional information such as class labels, spatial position, or track context may be needed.

These limitations mean that team identification should be treated as a useful but imperfect intermediate stage. Errors in team assignment can affect downstream outputs such as possession estimates, pass detection, and offside-related analysis, so temporal smoothing and special-case handling are important for improving the reliability of the full pipeline.

2.7 Pitch Registration and Planar Homography

Object detections and tracks are initially expressed in image coordinates, but tactical analysis usually requires positions in a common pitch coordinate system. For example, bird’s-eye visualisation, heatmaps, speed and distance estimation, possession analysis, and offside reasoning all depend on knowing where objects are located on the pitch rather than where they appear in the broadcast frame. Pitch registration addresses this problem by estimating a geometric mapping between the camera image and a canonical top-down model of the football pitch [44, 43].

2.7.1 Pitch Keypoint Detection

A football pitch contains a fixed set of semantic landmarks whose positions are known in a standard pitch model. Examples include line intersections, penalty-area corners, centre-circle points, and other distinctive points formed by pitch markings. If these landmarks can be detected in the broadcast image, they provide correspondences between image coordinates and pitch coordinates.

Let $\mathbf{x}_i = (u_i, v_i)^\top$ denote the image location of pitch landmark i , and let $\mathbf{X}_i = (x_i, y_i)^\top$ denote the corresponding point in the canonical pitch coordinate system. A set of such correspondences,

$$\{(\mathbf{x}_i, \mathbf{X}_i)\}_{i=1}^N,$$

can be used to estimate the transformation from image space to pitch space. In practice, the detected keypoints may also include a confidence score indicating how reliable each landmark prediction is.

Pitch keypoint detection can be viewed as a specialised form of pose estimation, where the target points are not human joints but fixed landmarks on the playing surface. Modern keypoint detectors use convolutional feature extraction and a prediction head to estimate the locations and confidence scores of predefined landmarks [17]. This is useful because it converts the registration problem into a supervised visual recognition task: the model learns to locate pitch landmarks from annotated examples.

The quality and distribution of the detected keypoints are important. At least four non-collinear correspondences are required to estimate a planar homography, but more points usually make the estimate more stable. Points spread across the visible pitch constrain the transformation better than points clustered in one small region. Incorrect or poorly localised keypoints can distort the estimated mapping, which is why robust estimation methods such as RANSAC are commonly used after keypoint detection.

2.7.2 Planar Homography

A planar homography is a projective transformation that maps points from one plane to another. In this project, it is used to map points from the broadcast image plane to a canonical top-down pitch plane. This is justified by the fact that the playing surface is approximately planar, so points lying on the pitch can be related by a single 3×3 transformation matrix [15].

Using homogeneous coordinates, a point in the image is written as

$$\tilde{\mathbf{x}} = (u, v, 1)^\top,$$

and its corresponding pitch-space point is written as

$$\tilde{\mathbf{X}} = (x, y, 1)^\top.$$

The homography H maps between them as

$$\tilde{\mathbf{X}} \sim H\tilde{\mathbf{x}}, \quad (2.16)$$

where $\sim$ denotes equality up to scale. The matrix H is defined only up to a non-zero scalar factor, so it has eight degrees of freedom rather than nine.

To estimate H , corresponding points are needed between the broadcast image and the pitch model. Since a homography has eight degrees of freedom, at least four non-collinear point correspondences are required. Each correspondence provides two independent equations, and with more than four correspondences the system becomes overdetermined. In that case, the homography is usually estimated in a least-squares sense using the Direct Linear Transform (DLT) algorithm [15].

Given correspondences $(\mathbf{x}_i, \mathbf{X}_i)$, DLT constructs a linear system

$$A\mathbf{h} = \mathbf{0}, \quad (2.17)$$

where $\mathbf{h}$ is the vectorised form of the homography matrix H . The solution is obtained by finding the vector $\mathbf{h}$ that minimises $\|A\mathbf{h}\|_2$ subject to $\|\mathbf{h}\|_2 = 1$. In practice, this can be solved using singular value decomposition, taking the right singular vector corresponding to the smallest singular value.

Once estimated, the homography allows image-space positions to be projected into pitch space. This is what makes it possible to compare player locations across frames

despite camera perspective, pan, tilt, and zoom. However, the mapping is only valid for points lying on or very close to the pitch plane. For player tracking, a common approximation is therefore to project the bottom centre of the player’s bounding box, since this point is closest to the player’s contact point with the ground.

2.7.3 Reprojection Error and Bird’s-Eye Mapping

Once a homography has been estimated, points on the pitch plane can be mapped from image coordinates into pitch coordinates. This produces a bird’s-eye representation in which player and ball locations can be analysed in a common coordinate system rather than in the perspective view of the broadcast camera. This is important because distances and relative positions in the image do not correspond directly to real pitch distances.

Registration quality is commonly measured using reprojection error. For a correspondence $(\mathbf{x}_i, \mathbf{X}_i)$, where $\mathbf{x}_i$ is the detected image point and $\mathbf{X}_i$ is its known pitch-space location, the reprojection error is

$$e_i = \|\mathbf{X}_i - \pi(H\tilde{\mathbf{x}}_i)\|_2, \quad (2.18)$$

where H is the estimated homography, $\tilde{\mathbf{x}}_i$ is the homogeneous form of the image point, and $\pi(\cdot)$ converts homogeneous coordinates back to Cartesian coordinates. The mean reprojection error over N correspondences is then

$$\bar{e} = \frac{1}{N} \sum_{i=1}^N e_i. \quad (2.19)$$

A low reprojection error indicates that the estimated homography maps the detected landmarks close to their expected pitch locations. However, this does not guarantee that all projected player positions are accurate. The estimate can still be unreliable if the keypoints are poorly distributed, if too few landmarks are visible, or if the detected points are affected by localisation error. For this reason, reprojection error should be interpreted alongside qualitative inspection of the resulting bird’s-eye mapping.

After registration, tracked player positions can be projected onto the pitch by applying the homography to an image-space point associated with each player. A common approximation is to use the bottom centre of the player’s bounding box, since this point is closest to the player’s contact point with the ground. These projected positions form the basis for pitch-space outputs such as bird’s-eye visualisations, heatmaps, speed and

distance estimates, possession analysis, and offside-related reasoning [44, 43].

2.8 Spatial and Tactical Analysis in Pitch Space

Once detections and tracks have been mapped onto a canonical pitch model, analysis can be performed in a coordinate system that is independent of the broadcast camera view. This is important because measurements in image pixels are affected by perspective: two equal pixel distances may correspond to different real-world distances depending on where they occur in the frame. Pitch-space coordinates make it possible to compare player positions, estimate movement statistics, construct heatmaps, and reason about tactical events such as possession, passes, and offside decisions [44, 12].

2.8.1 Player Projection onto the Pitch

To analyse player movement on the pitch, each detected player must be represented by a single image point before applying the homography. A common choice is the bottom-centre of the bounding box, since this point is closest to the player’s contact point with the ground. Using the centre of the bounding box would place the point above the pitch plane, which can introduce a systematic projection error because the homography is valid for points lying on the ground plane.

Let $B_t^{(i)}$ denote the bounding box of player i at frame t , and let $\mathbf{q}_t^{(i)} \in \mathbb{R}^2$ denote its bottom-centre point in image coordinates. Given an image-to-pitch homography H , the projected pitch-space position is

$$\mathbf{p}_t^{(i)} = \pi\left(H\tilde{\mathbf{q}}_t^{(i)}\right), \quad (2.20)$$

where $\tilde{\mathbf{q}}_t^{(i)}$ is the homogeneous form of the image point and $\pi(\cdot)$ converts homogeneous coordinates back to Cartesian coordinates.

The sequence

$$\{\mathbf{p}_t^{(i)}\}_{t=1}^T$$

forms the pitch-space trajectory of player i . These trajectories provide the geometric basis for later spatial analysis: heatmaps use the distribution of positions over time, speed and distance use changes in position between frames, and tactical reasoning uses the relative positions of players, the ball, and pitch landmarks.

2.8.2 Heatmap Construction

A heatmap summarises where a player or team spends time on the pitch. After player detections have been projected into pitch coordinates, the pitch can be divided into a regular grid and each projected position can be assigned to a grid cell. Over a sequence of frames, cells visited more frequently receive higher values, producing a spatial distribution of player activity.

For player i , let $\mathbf{p}_t^{(i)}$ be the projected pitch-space position at frame t . A simple occupancy heatmap can be defined as

$$H_i(u, v) = \sum_{t=1}^T \mathbf{1} \left[(u, v) = \text{bin} \left(\mathbf{p}_t^{(i)} \right) \right], \quad (2.21)$$

where $\mathbf{1}[\cdot]$ is the indicator function and $\text{bin}(\cdot)$ maps a continuous pitch coordinate to a discrete grid cell. The same idea can be applied to a whole team by accumulating positions from all players assigned to that team.

Raw occupancy maps can be sparse, especially when a sequence is short or tracking is fragmented. To make the spatial pattern easier to interpret, the grid can be smoothed using a Gaussian kernel:

$$\tilde{H}_i = H_i * G_\sigma, \quad (2.22)$$

where $*$ denotes convolution and σ controls the amount of spatial smoothing. A larger value of σ produces a smoother map but can hide fine-grained positional detail.

Heatmaps are useful for interpreting broad positional behaviour, such as whether a full-back stays wide, whether a midfielder occupies central zones, or whether a team attacks more frequently on one side. However, they depend directly on the quality of the projected trajectories. Homography errors can shift positions to the wrong pitch region, while tracking identity switches can mix the movement of different players into the same heatmap.

2.8.3 Speed and Distance Estimation

Once player positions are available in pitch coordinates, movement statistics can be estimated from changes in position over time. Let f_s denote the video frame rate, so that the time between consecutive frames is

$$\Delta t = \frac{1}{f_s}.$$

For player i , a finite-difference estimate of velocity at frame t is

$$\mathbf{v}_t^{(i)} = \frac{\mathbf{p}_t^{(i)} - \mathbf{p}_{t-1}^{(i)}}{\Delta t}, \quad (2.23)$$

where $\mathbf{p}_t^{(i)}$ and $\mathbf{p}_{t-1}^{(i)}$ are consecutive pitch-space positions. The corresponding instantaneous speed is

$$s_t^{(i)} = \left\| \mathbf{v}_t^{(i)} \right\|_2. \quad (2.24)$$

Total distance covered can then be approximated by summing the Euclidean displacement between consecutive positions:

$$d^{(i)} = \sum_{t=2}^T \left\| \mathbf{p}_t^{(i)} - \mathbf{p}_{t-1}^{(i)} \right\|_2. \quad (2.25)$$

This assumes that the pitch-space coordinates are expressed in real-world units, such as metres. If the pitch model is defined in normalised coordinates, an additional scaling step is required before interpreting distances physically.

Speed and distance estimates are sensitive to noise in the projected trajectories. Small localisation errors can be amplified when positions are differentiated between frames, producing unrealistic speed spikes. Tracking identity switches can also create artificial jumps in position, causing distance to be overestimated. For this reason, trajectory smoothing and plausibility checks are commonly applied before computing movement statistics. In broadcast-derived pipelines, this is especially important because the projected positions are less accurate than those obtained from dedicated tracking systems [3, 25].

2.8.4 Possession and Pass Detection

Possession and passes are not detected directly from a single video frame. They are inferred from the spatial relationship between the ball, players, and team identities over time. Once the ball and players have been projected into pitch coordinates, a simple possession estimate can be made by assigning the ball to the nearest eligible player within a maximum distance threshold.

Let $\mathbf{b}_t \in \mathbb{R}^2$ denote the pitch-space position of the ball at frame t , and let $\mathbf{p}_t^{(i)} \in \mathbb{R}^2$ denote the pitch-space position of player i . For a set of candidate players $\mathcal{A}_t$, the nearest

player to the ball is

$$h_t = \arg \min_{i \in \mathcal{A}_t} \left\| \mathbf{b}_t - \mathbf{p}_t^{(i)} \right\|_2. \quad (2.26)$$

If the distance between the ball and this player is greater than a chosen threshold $d_{\max}$, possession can be left unassigned. This prevents the system from forcing a possession label when the ball is airborne, far from all players, or poorly localised.

A pass can then be treated as a stable transition in possession from one player to another, usually between players on the same team. For example, if player i is assigned possession for several frames and the ball is later assigned to player j , this may indicate a pass from i to j , provided the transition is not caused by a short localisation error or a contested ball. Temporal stability is therefore important: raw frame-level possession can flicker because of noisy ball detections, player occlusion, or small projection errors.

More advanced approaches model possession using player influence or pitch-control surfaces rather than nearest-player distance [11]. These methods can capture richer tactical context, but they require reliable player velocities, orientations, and high-quality tracking data. For this project, the relevant theoretical point is that possession and pass detection are downstream inferences over projected trajectories rather than direct detector outputs. Their reliability therefore depends on the accuracy of ball detection, player tracking, team identification, and pitch registration [12].

2.8.5 Offside Rule Formalisation

Offside is a spatial rule, so it is naturally suited to pitch-space reasoning. Under Law 11 of the Laws of the Game, a player is in an offside position if, at the moment the ball is played by a team-mate, they are in the opponents' half and nearer to the opponents' goal line than both the ball and the second-last opponent [42].

For an automated system, this can be simplified into a positional offside test. Let $x_t^{(a)}$ denote the pitch-space coordinate of an attacking player along the attacking direction, $x_t^{(b)}$ the corresponding ball coordinate, and $x_t^{(o_2)}$ the coordinate of the second-last opponent. A player is in a positional offside position if

$$x_t^{(a)} > \max \left(x_t^{(b)}, x_t^{(o_2)} \right), \quad (2.27)$$

and the player is in the opponents' half. The use of the maximum expresses the requirement that the attacker must be beyond both the ball and the second-last opponent.

This formulation depends on knowing the attacking direction of each team. If the attacking direction is reversed, the comparison in Equation (2.27) is inverted and the offside decision becomes invalid. It also depends on the moment at which the ball is played, which must be approximated from the possession or pass-detection sequence.

The formulation above should therefore be interpreted as positional offside detection rather than a complete implementation of Law 11. A complete offside decision also depends on whether the player becomes actively involved in play, which cannot be determined reliably from simple trajectories alone [42]. Nevertheless, the rule provides a useful example of how pitch-space geometry can support tactical reasoning once players, teams, and the ball have been registered to a common coordinate system.

2.9 Error Propagation in a Multi-Stage Pipeline

A broadcast-football analysis system is a multi-stage pipeline in which later outputs depend on intermediate estimates produced by earlier components. Errors therefore do not remain isolated within the component where they occur. A detection error can affect tracking; a tracking error can affect team assignment and movement statistics; and a registration error can affect every pitch-space output. This is an important consideration in sports-video pipelines because detection, tracking, pitch registration, and event reasoning are tightly connected [44, 43].

At an abstract level, the pipeline can be viewed as a composition of processing stages:

$$y = f_n(f_{n-1}(\cdots f_2(f_1(x))\cdots)), \quad (2.28)$$

where x is the input video, each f_k represents one stage of the pipeline, and y is a final output such as a heatmap, possession estimate, or offside decision. In this project, these stages include object detection, multi-object tracking, team identification, pitch registration, and pitch-space tactical reasoning.

Figure 2.7 illustrates the main dependency structure. Several error paths are especially important. A missed player detection may break a track, reducing the reliability of heatmaps, speed estimates, or team assignment. An identity switch can artificially inflate distance covered or create false changes in possession. A homography error can shift projected player positions, distorting heatmaps and changing the relative ordering of attackers and defenders in offside analysis. Similarly, an incorrect team label can affect possession, pass detection, and attacking-direction inference even when the underlying player tracks are accurate.

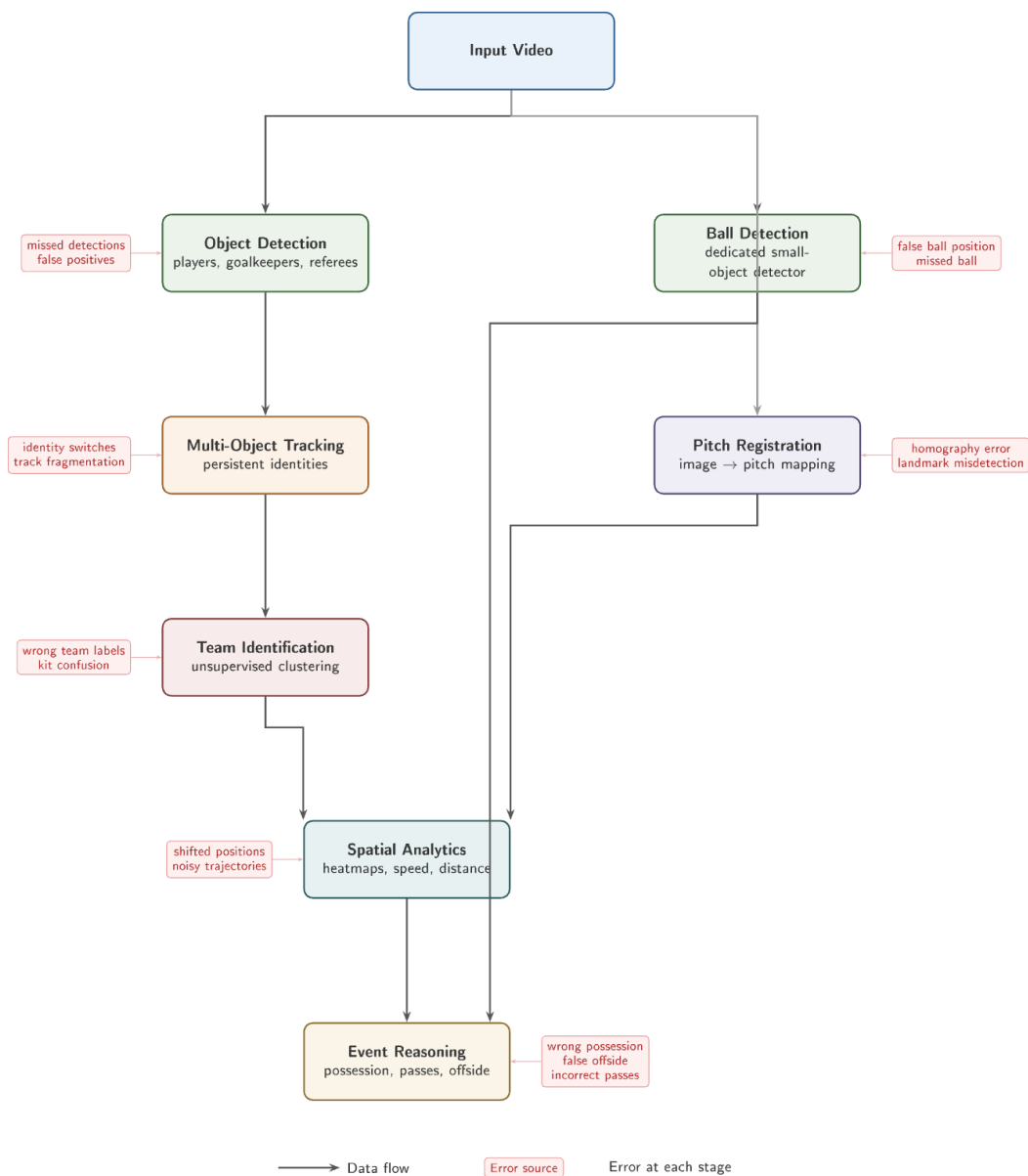


Figure 2.7: Error propagation paths in a broadcast football analysis pipeline. Solid arrows show the main data flow between processing stages, while dashed arrows indicate how errors at each stage can affect downstream outputs.

This means that downstream outputs cannot be interpreted independently of upstream quality. A poor offside decision may result from weak event logic, but it may also originate from earlier errors in ball detection, player tracking, team assignment, or pitch registration. Similarly, noisy speed estimates may be caused by projection error or identity fragmentation rather than by the speed-estimation formula itself.

For this reason, evaluation should consider both individual components and the integrated pipeline. Component-level metrics help identify where errors originate, while system-level evaluation shows whether the combined outputs remain useful for tactical interpretation [12]. This motivates the evaluation strategy used in Chapter 5.

2.10 Chapter Summary

This chapter has established the theoretical foundation for the main components of the broadcast football analysis pipeline. It first introduced the machine learning and computer vision concepts needed to understand the later methods, before reviewing related work in commercial tracking systems, academic benchmarks, and open broadcast-video pipelines. The chapter then covered object detection, multi-object tracking, team identification, pitch registration, spatial and tactical analysis, and error propagation.

The chapter has also introduced the main formalisms and metrics used throughout the report, including bounding boxes, IoU, precision, recall, mAP, Kalman filtering, data association, HOTA, visual embeddings, k -means clustering, planar homography, reprojection error, heatmaps, speed and distance estimation, possession reasoning, and positional offside detection. These concepts provide the basis for the implementation described in Chapter 3 and the evaluation presented in Chapter 5.

A central theme of the chapter is that the pipeline is modular but interdependent. Each stage has a clear input–output role, which makes the system easier to describe, implement, and evaluate. However, errors can propagate between stages: missed detections can affect tracking, tracking errors can affect team assignment and movement statistics, and homography errors can affect all pitch-space outputs. The following chapter builds on this theory by describing how these components are implemented in the proposed system.

Chapter 3

Methodology and Implementation

3.1 Chapter Overview

Chapter 2 surveyed the methods relevant to each stage of a broadcast football analysis pipeline—object detection, tracking, team identification, pitch registration, and the derivation of tactical outputs—without committing to a specific realisation. This chapter moves from that theoretical grounding to the concrete system developed for this project, describing the architecture and implementation of each stage and the reasoning behind the choices made at each one.

The pipeline takes a single broadcast video stream as input and processes it through a sequence of stages: player, goalkeeper, referee and ball detection with dedicated YOLO models; multi-object tracking by association; team identification via SigLIP embeddings of torso crops followed by dimensionality reduction and clustering; pitch registration from keypoint correspondences resolved into a planar homography; and projection of tracked players into a pitch-coordinate frame from which heatmaps, speed and distance, possession, and offside geometry are derived. Each stage consumes the output of those preceding it, and the chapter follows this same ordering so that each design decision can be read in the context of the inputs it must handle and the outputs it must produce.

For each stage, the chapter justifies the design choice against the alternatives surveyed in Chapter 2, specifies parameter values and algorithmic details in sufficient detail for a peer to reproduce the work, and presents illustrative outputs from the implemented system. The datasets used for training the bespoke detection and keypoint models are described, together with the training procedure and the resulting model artefacts. The chapter also defines the evaluation methodology that will be applied in

Chapter 5, setting out the metrics, test data, and protocols by which each pipeline stage and the system as a whole will be assessed.

Chapter 4 presents the results obtained by running the pipeline on evaluation data, and Chapter 5 discusses those results and their implications; the present chapter is concerned with how the system was built and why, not with its measured performance.

3.2 Requirements and Design Objectives

Before the system architecture is presented, this section sets out the requirements the pipeline is designed to meet. Functional requirements define what the system must do; non-functional requirements define the qualities it should exhibit; and success criteria state the measurable thresholds against which each component will be judged in Chapter 5. The requirements derive from the project aims set out in Chapter 1 and are framed here in terms suited to system design, so that the architectural and implementation choices described in the remainder of this chapter can be read against them.

3.2.1 Functional Requirements

The functional requirements describe the end-to-end behaviour of the pipeline. The system takes as input a single broadcast football video and produces, for each frame, a set of per-object detections, persistent track identities, team labels, pitch-coordinate positions, and a collection of derived tactical outputs. The requirements are decomposed by pipeline stage so that each subsequent section of this chapter can be mapped directly to the requirement it satisfies.

FR1: Object detection. The system shall detect the four classes relevant to broadcast football analysis—players, goalkeepers, referees, and the ball—in each frame of the input video, returning axis-aligned bounding boxes and class labels. Given the extreme scale disparity between outfield players and the ball, detection of the ball shall be handled by a dedicated model operating at the same input resolution but with its own training regime and confidence regime, rather than relying on the generic detector alone.

FR2: Multi-object tracking. The system shall maintain consistent identities for detected players, goalkeepers, and referees across frames, associating detections over

time into persistent tracks. Tracker choice shall be configurable so that alternative association strategies can be compared. Ball identity is not required to persist, since the ball is a single object and its per-frame localisation is sufficient for downstream use.

FR3: Team identification. The system shall assign each tracked player to one of two teams without manual labels, using visual appearance alone. The assignment shall be stable over time: transient misclassification on a single frame shall not be allowed to propagate as the team label returned for a track. Goalkeepers, whose kits typically differ from their outfield teammates, shall be resolved to a team by a separate mechanism rather than being forced into the outfield clustering.

FR4: Pitch registration. The system shall recover the planar homography between the broadcast image and a canonical pitch model, permitting image-plane coordinates to be mapped to pitch-plane coordinates. Registration shall tolerate the fact that the full pitch is rarely visible in a broadcast frame, and shall degrade gracefully when insufficient correspondences are available in a given frame by reusing the most recent valid transform until a new valid estimate is obtained.

FR5: Tactical output generation. From the projected per-track trajectories the system shall derive a set of tactical outputs: per-player heatmaps of pitch occupation, speed and distance statistics, team possession, pass events, and offside candidate frames. Each output shall be produced in pitch coordinates, so that the reported quantities are physically meaningful rather than tied to the broadcast view.

FR6: Visualisation and persistence. The system shall render the principal outputs for inspection—annotated broadcast video with detection, track, and team overlays; a bird’s-eye projection of tracked objects onto the pitch model; and rendered heatmaps—and shall persist these together with any per-frame numerical outputs so that results can be reviewed offline and used as input to the evaluation procedure.

3.2.2 Non-functional Requirements

The non-functional requirements describe qualities the system should exhibit irrespective of the specific outputs it produces.

NFR1: Reproducibility. The full configuration required to reproduce a given run—model weights, parameter values, and preset selections—shall be recoverable from a single set of launch arguments, and the training of any bespoke models shall be recorded with sufficient detail (dataset, hyperparameters, training curves) to be repeated by a peer.

NFR2: Modularity. Each pipeline stage shall be implemented as a self-contained component with a clearly defined input and output contract, so that an individual stage can be replaced, reconfigured, or evaluated in isolation without requiring changes to the surrounding system.

NFR3: Configurability. Parameter values that govern behaviour at runtime—detection thresholds, tracker selection, inference strides, warmup sizes, smoothing windows, displacement gates—shall be exposed through the command-line interface rather than embedded in source code, and reasonable defaults shall be provided in a single location so that the system can be run without a long argument list.

NFR4: Robustness to broadcast artefacts. The pipeline operates on broadcast video rather than instrumented capture, and must therefore tolerate partial occlusion, motion blur, pan-tilt-zoom camera motion, and the appearance of on-screen graphics without catastrophic failure. Robustness is treated as a design objective at each stage rather than as a single global property, with appropriate mechanisms (temporal smoothing, displacement gates, majority-vote filtering) introduced where individual stages are known to be sensitive.

NFR5: Graceful degradation. When an individual stage cannot produce a usable output for a given frame, the pipeline shall continue to operate on subsequent frames rather than terminate. For pitch registration in particular, the system shall retain the most recent valid homography and reuse it until a new valid estimate is produced. The extent to which other stages meet this objective is discussed alongside the relevant component.

NFR6: Inspectability. Intermediate outputs of the pipeline—raw detections, tracked identities, projected positions, registration overlays, per-track statistics—shall be explicable for inspection, so that failure modes can be localised to specific stages during

development and evaluation, and so that the evaluation procedure can assess each stage independently.

NFR7: Runtime. The pipeline is intended for offline post-match analysis rather than real-time or live operation. Runtime is therefore not a primary design objective: the system shall complete processing of a broadcast clip on a single consumer GPU within a reasonable working session, but no throughput target is set, and design decisions throughout the pipeline prioritise per-stage accuracy and inspectability over end-to-end speed. The measured processing cost and the stages that dominate it are discussed alongside the results in Chapter 5.

3.2.3 Success Criteria

The success criteria restate, in measurable form, the conditions under which each functional requirement is considered satisfactorily met. Specific numerical thresholds are stated where they derive from the project aims set in Chapter 1; the full evaluation protocols by which these are assessed are given in Section 3.10 and applied in Chapter 5.

SC1 (FR1, Detection). The generic object detector shall achieve a mean average precision of at least on the held-out test split, with per-class performance reported for player, goalkeeper, and referee. The dedicated ball detector shall achieve a per-frame recall of at least under the confidence regime used at inference, at a false-positive rate that the temporal-filtering stage (Section ??) can reasonably suppress.

[target
mAP@0.5
from
Ch1]

SC2 (FR2, Tracking). Tracking performance shall be reported using the HOTA family of metrics [29], with a target HOTA of at least on the evaluation clips. Identity switches shall be reported alongside and kept below per minute of broadcast time.

[target]

[target]

[target]

SC3 (FR3, Team identification). The per-frame team assignment shall agree with manually annotated ground truth on at least of sampled player crops, and the per-track label returned after temporal smoothing shall agree on at least of tracks. Goalkeeper assignment shall be correct on at least of goalkeeper tracks.

[target]

[target]

[target]

SC4 (FR4, Registration). Pitch registration shall achieve a mean reprojection error of no more than on evaluation frames where ground-truth correspondences are available, and shall produce a valid homography on at least of frames in a continuous

[target,
in
pitch
me-
tres]

[target]

broadcast clip.

SC5 (FR5, Tactical outputs). Tactical outputs shall be plausible when inspected against the broadcast source: heatmaps shall reflect known player roles, speed and distance values shall fall within physically reasonable ranges reported in the sports-science literature, possession totals shall be consistent with a human annotator’s reading of the clip, detected pass events shall match annotated passes with a precision and recall of at least , and offside candidate frames shall agree with a human referee’s reading.

[target]

SC6 (End-to-end usefulness). When run end-to-end on unseen broadcast footage, the system shall produce the full set of visualisations and statistics described under FR6 without manual intervention, and the resulting outputs shall be informative enough that a non-specialist viewer can interpret the match in tactical terms from the produced artefacts alone.

3.3 System Architecture

This section describes the architecture of the pipeline at a level that permits the work to be reproduced by a peer. The pipeline is presented first as a structural overview, showing how the stages are composed and where models, rule-based logic, and caching enter the design (Section 3.3.1). The data objects passed between stages are then specified with their types, shapes, and provenance (Section 3.3.2), and the user-facing outputs of the system are catalogued (Section 3.3.3). The algorithmic content of each stage is deferred to Sections 3.6 through ??, which describe how the stages are realised and why.

3.3.1 Pipeline Overview

Figure 3.1 shows the end-to-end architecture. A single broadcast video frame is processed through two parallel branches that converge at a pitch-space projection stage, after which all higher-level analytics operate in a shared pitch coordinate frame rather than in the image plane.

The left branch handles *what* is in the frame: objects are detected, outfield actors are tracked over time, and each tracked player is assigned to a team. The right branch

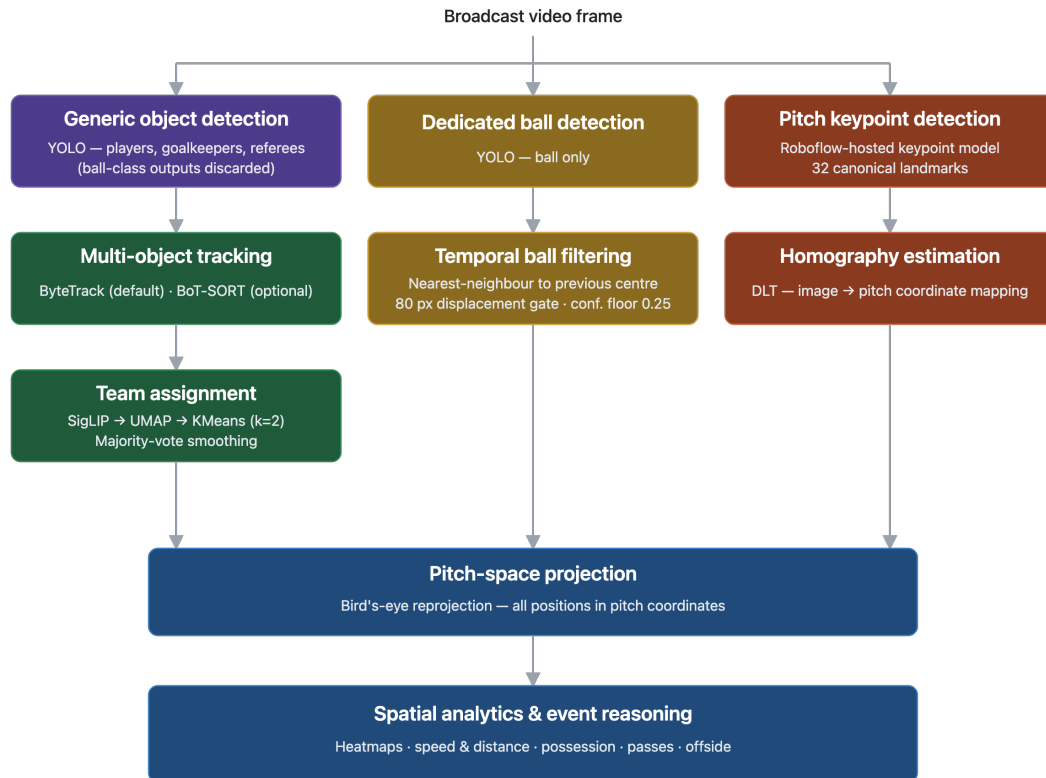


Figure 3.1: End-to-end pipeline architecture. The broadcast frame enters at the top and is processed through two parallel branches—appearance-based detection and tracking on the left, pitch-geometry recovery on the right—which converge at the pitch-space projection stage. All higher-level tactical analytics operate in pitch coordinates. Section numbers on the left indicate where each stage is described in detail.

handles *where* the frame sits in pitch-space: canonical landmarks on the pitch are detected and resolved into a planar homography. The two branches are independent—no stage in one depends on an output of the other until they converge—and this independence is exploited at runtime, since pitch keypoints change slowly relative to player positions and the right branch is therefore executed at a lower frequency than the left.

Models and rule-based components. The appearance stages are fully model-based. A generic YOLO detector is run on each frame and produces bounding boxes for players, goalkeepers, and referees; a second YOLO detector dedicated to the ball is run in parallel on the same frame and is the authoritative source of ball localisation

(the ball-class outputs of the generic detector are discarded, for reasons given in Section 3.6). Pitch landmarks are produced by a keypoint model. The team identification stage combines a pretrained vision-language embedding model (SigLIP) with dimensionality reduction (UMAP) and a small unsupervised clustering step (k-means, $k = 2$). The tracker is a learning-free multi-object association algorithm (ByteTrack, with BoT-SORT selectable) applied to the generic detector’s player, goalkeeper, and referee outputs only; the ball is never passed to the tracker and instead passes through a lightweight temporal filter (nearest neighbour to the previous accepted position under a displacement gate) described in Section 3.6. All stages downstream of projection—heatmap accumulation, speed and distance computation, possession, pass detection, offside geometry—are rule-based: they operate deterministically on projected trajectories and carry no learned parameters.

Caching and stride. Two caching mechanisms are used to control the per-frame computational cost. First, pitch keypoint inference is run once every `pitch-stride` frames (default 15) and the resulting homography is reused on intervening frames; when a new inference fails to produce a valid estimate, the last valid homography is retained. Second, team classification is fitted once during a warmup window at the start of the video (default 20 s in the production preset) and reused thereafter; per-track team labels are produced by applying the fitted classifier to new crops and are stabilised by a majority-vote smoother over a 30-frame window. No other stage caches state across frames beyond the track state held internally by the tracker.

Execution model. The pipeline is driven by a single orchestration loop in `src/pipeline/run.py` that reads frames sequentially, invokes the stages in the order shown in Figure 3.1, and writes outputs to disk. The loop is deliberately sequential rather than concurrent: the stages have strict data dependencies, intermediate outputs are required for inspection and evaluation (NFR6), and real-time operation is explicitly out of scope (NFR7).

3.3.2 Data Flow and Intermediate Representations

The data objects passed between stages are summarised in Table 3.1. Where possible the pipeline reuses the `supervision` library’s native `Detections` type as the common currency of the detection–tracking–team-identification branch, which simplifies inter-operation with the off-the-shelf tracker and annotation utilities. The cost of this choice

Object	Fields	Producer	Consumer(s)
Raw detection set	xyxy, confidence, class_id	Detector (§3.6)	Tracker
Tracked detection set	+ tracker_id	Tracker (§3.6)	Team ID, projection
Team-labelled detection set	class_id over-written	Team ID (§??)	Projection, analytics
Ball detection (filtered)	xyxy, confidence	Ball detector + filter (§3.6)	Projection, analytics
Ball detections (generic)	xyxy, confidence, tracker_id	Generic detector + tracker	<i>Discarded</i>
View transformer	$H \in \mathbb{R}^{3 \times 3}$	Pitch registration (§??)	Projection
Projected positions	pitch coords (cm)	Projection (§??)	Heatmaps, stats, events
Event record	(t, τ, I)	Event reasoning (§??)	Logs, exports

Table 3.1: Data objects passed between pipeline stages. The `supervision.Detections` type is used throughout the left branch of the pipeline with fields progressively populated and, in the case of `class_id`, reinterpreted.

is that per-stage metadata that does not fit the `Detections` schema must be handled out of band, as described below.

Detection and tracking record. The core data object in the left branch is `supervision.Detections` with four populated fields. A raw detection set $\mathcal{D}$ of N objects carries $\mathbf{B} \in \mathbb{R}^{N \times 4}$ for bounding boxes in absolute pixel coordinates (x_1, y_1, x_2, y_2) , $\mathbf{c} \in [0, 1]^N$ for per-detection confidences, and $\mathbf{k} \in \{0, 1, 2, 3\}^N$ for class identifiers under the detector’s label set $\{0:\text{ball}, 1:\text{GK}, 2:\text{player}, 3:\text{referee}\}$. The tracker additionally populates $\mathbf{t} \in \mathbb{Z}^N$ with integer track identifiers on frames where association has succeeded; on frames where the tracker has produced no identifier, this field is absent.

Encoding of team membership. Team membership is encoded by overwriting the `class_id` field in place rather than by adding a new field to the detection record. Following team assignment, the semantics of `class_id` change: outfield players carry `class_id` $\in \{0, 1\}$ denoting team A or team B, goalkeepers are resolved to the same two-valued alphabet where possible and to the value 2 otherwise, and referees are

forced to 2. This reinterpretation is deliberate: downstream renderers and exporters consume the field uniformly as a palette index, and the reuse avoids maintaining two parallel arrays that must be kept in lockstep. It is documented here explicitly because the value 2 denotes *referee or unresolved goalkeeper* after team assignment, not *player* as in the raw detector output.

Ball as a distinct data stream. The ball is handled as a separate `supervision.Detections` instance throughout the pipeline. It is never merged with the outfield detection set, never carries a `tracker_id`, and is consumed by downstream stages (bird’s-eye render, offside detector, stats tracker) as a parallel argument alongside the outfield detections. On frames where the temporal filter rejects all candidates, the ball set is empty; downstream consumers treat this condition as a missing-ball frame rather than as an error. The rationale for this parallel-stream design is given in Section 3.6.

Pitch geometry and projected coordinates. Pitch geometry is carried on a separate `ViewTransformer` object rather than attached to detections. The transformer encapsulates the 3×3 homography H from broadcast-image pixels to a canonical pitch plane expressed in centimetres. Downstream consumers that require pitch-space positions (heatmap accumulation, speed and distance computation, possession, pass detection, offside geometry) receive the detection set and the transformer as separate arguments and compute $\tilde{\mathbf{p}} = H \cdot [x, y, 1]^\top$ internally from the bottom-centre anchor of each bounding box, applying the standard perspective divide. This out-of-band handling mirrors the fact that geometry changes on a different schedule from appearance: a single transformer is used across all detections in a frame, and often across multiple frames under the keypoint-stride caching scheme.

Event records. Higher-level outputs produced by the analytics stage (possession segments, pass events, offside candidate frames) are represented as discrete records of the form (t, τ, I) , where t is a frame timestamp, τ is an event type, and I is a reference to the tracks and teams involved in the event. Records are accumulated across the video and exported as structured logs alongside the rendered visualisations.

Persisted artefacts. Each run produces a deterministic set of on-disk artefacts under a timestamped output directory: the annotated broadcast video, the bird’s-eye render (when enabled), the side-by-side composite (when enabled), per-track heatmap images, per-track movement summaries, a possession timeline, a pass event log, and a

flagged-frames list for offside candidates. These artefacts together form the input to the evaluation procedure described in Section 3.10 and are the basis of the results reported in Chapter 5.

3.3.3 System Outputs

The system produces three families of outputs, corresponding to the three principal ways the results are consumed: inspection of the broadcast through augmented video, compact summary through bird’s-eye visualisation, and analytical use through numerical exports.

Augmented broadcast. The annotated broadcast video renders per-frame detections, persistent track identifiers, and team colour coding back onto the source frame. This artefact is the most direct qualitative evidence that the pipeline is operating correctly: detection misses, identity switches, and team misassignments are immediately visible against the broadcast action.

Bird’s-eye visualisation. The bird’s-eye render projects every tracked outfield object, along with the ball, onto a scale drawing of the pitch. Objects are drawn as coloured discs whose colour encodes the (post-team-assignment) class identifier: two team colours for outfield players and resolved goalkeepers, a distinct colour for referees and unresolved goalkeepers, and a neutral colour for the ball. When an offside candidate has been flagged on a given frame, the offending player is recoloured and the relevant offside line is overlaid. The render is stateless: each frame is drawn from the current set of projected positions alone, without trails or accumulated history, which keeps the bird’s-eye faithful to the instantaneous pitch state. The bird’s-eye can be produced as a standalone video, as a component of a side-by-side composite with the broadcast, or both simultaneously.

Tactical outputs and numerical exports. The tactical analytics produce a set of quantitative outputs intended for post-hoc analysis rather than inline display. Per-player heatmaps capture occupation density in pitch space. Per-player movement summaries record distance covered and speed statistics over the sequence. A possession timeline records which team is in possession at each moment. Pass events and offside candidate frames are accumulated as event records. Each output is persisted in a structured, machine-readable form (CSV or JSON as appropriate to the shape of the data)

Model	Classes	Train	Val	Test
Generic detector	ball, goalkeeper, player, referee	618	75	75
Ball detector	ball (single class)	932	114	119
Pitch keypoints	pitch (+ 32 keypoints)	~290	34	28

Table 3.2: Summary of the three training datasets. All datasets are hosted on Roboflow Universe and were downloaded via the Roboflow Python SDK; splits are those provided by the dataset version on Roboflow.

so that the evaluation procedure can compare against ground-truth annotations without re-running the pipeline, and so that the outputs can be consumed by downstream tools outside the scope of this project.

3.4 Datasets and Model Training

Three bespoke models are used in the pipeline: a generic object detector for outfield actors, a dedicated ball detector, and a pitch keypoint detector. Each was trained on a Roboflow-hosted dataset of broadcast football imagery, using the Ultralytics training harness with variants of the YOLOv8 architecture. This section describes the datasets (§3.4.1) and the training procedure for each model (§3.4.2, §3.5.4, §3.5.5). Results on held-out training splits are reported alongside the pipeline stage that consumes each model, where they inform the design discussion directly; full pipeline-level evaluation appears in Chapter 5. A fourth model used by the pipeline—the SigLIP vision–language encoder consumed by the team identification stage—is used off-the-shelf without fine-tuning and is described in Section ?? rather than here.

3.4.1 Datasets

All three training datasets were curated under a single Roboflow Universe workspace, were downloaded via the Roboflow Python SDK into the training environment, and were exported in YOLO format with the standard train/validation/test splits handled by Roboflow’s dataset versioning. Table 3.2 summarises the three datasets.

Generic detector dataset. The generic detector was trained on a Roboflow-hosted dataset of broadcast football frames curated collaboratively as a shared resource for broadcast football analysis, to which the author contributed additional annotations of frames sourced from a publicly available broadcast football dataset hosted on Kaggle.

The resulting dataset contains 768 annotated frames split 618/75/75 for training, validation, and test, and carries the four classes required by this work: *ball*, *goalkeeper*, *player*, and *referee*. The class distribution is heavily imbalanced: the *player* class accounts for the great majority of annotations, while goalkeepers, referees, and the ball each appear at most once per frame and sometimes not at all. This imbalance motivates the use of a separate, single-class dataset for the ball, described next.

Ball detector dataset. The dedicated ball detector was trained on a separate Roboflow-hosted dataset built specifically for small-object detection of the ball against broadcast backgrounds. It contains 1,165 annotated frames split 932/114/119 for training, validation, and test, with a single *ball* class; the Ultralytics log records the model’s class head being overridden from 80 to 1 on initialisation. Sourcing a ball-only dataset rather than extracting ball annotations from the generic dataset yields a substantially larger pool of positive examples (932 versus 618 frames) and permits augmentation, input resolution, and loss weighting to be tuned for a single small-object class without having to balance against the needs of the outfield classes.

Pitch keypoint dataset. The pitch keypoint model was trained on a Roboflow-hosted dataset of broadcast football frames annotated with 32 canonical pitch landmarks together with a pitch bounding box. The dataset is comparatively small (approximately 290 training frames, 34 validation, 28 test). Two factors make this size acceptable: keypoint annotation is considerably more labour-intensive than bounding-box annotation, and the geometric regularity of a football pitch—fixed markings at fixed distances—means that a relatively small dataset can cover the space of camera viewpoints encountered in broadcast footage. The keypoint layout defines a horizontal-flip symmetry (`flip_idx`) so that horizontal augmentation can be applied without destroying left-right keypoint identities.

3.4.2 Generic Object Detector Training

The generic detector was initialised from the pretrained `yolov8m.pt` checkpoint and fine-tuned for 50 epochs on the four-class dataset. Training was run on a Tesla T4 GPU in a Google Colab environment, at an input resolution of 1280×1280 to preserve fine detail on small or distant players, with a batch size of 4 constrained by GPU memory at that resolution. The Ultralytics `auto` optimiser selected AdamW with default learning-rate schedules. Augmentation settings were the Ultralytics defaults for a detection

task (HSV jitter, mosaic, horizontal flip, random erasing, and RandAugment), with rotation, shear, and perspective augmentation disabled as inappropriate for a broadcast view with a largely fixed horizon.

Artefact retention. The final weights from this run were deployed to Roboflow for downstream use and retrieved from that deployment for pipeline inference. The `per-epoch results.csv` from the training run was not preserved in the project repository, so loss and mAP trajectories over training cannot be reconstructed. The final validation and test summaries emitted at the end of training are retained in the training notebook and are reported in §3.6, where they inform the design choice of treating the ball as a separate detection stream. This is a reproducibility limitation that the remaining two training runs, reported below, avoid.

3.5 Small-Object Detection and Dedicated Ball Detection

Although the ball is the primary object of interest in football, it is also the hardest to detect reliably in broadcast footage. At typical broadcast resolution, the ball subtends only a few dozen pixels in diameter, moves rapidly enough to produce motion blur, and is frequently occluded by players or absent from the frame entirely [14]. These properties place it firmly in the small-object detection regime, where standard detectors trained jointly on all classes tend to underperform because larger, more visually prominent objects dominate the training signal [44].

3.5.1 Why Small Objects Are Harder to Detect

The COCO benchmark defines a small object as one whose bounding box occupies fewer than 32×32 pixels [24]. By this standard, the ball in a wide-angle broadcast frame is consistently small, often below 20×20 pixels depending on camera zoom.

Small-object detection is inherently harder for two reasons. First, the IoU criterion (Equation (2.2)) becomes disproportionately sensitive to localisation error at small scales: a fixed pixel displacement δ produces a much larger IoU drop on a small box than on a large one. As shown in Figure 3.2, a 2-pixel shift reduces IoU from 1.0 to 0.78 on a 16×16 box but only to 0.94 on a 64×64 box. This makes ball detection

acutely sensitive to blur, noise, and detector uncertainty. Second, in convolutional architectures the spatial resolution of feature maps decreases with depth; by the deepest layers, small objects may occupy only a single cell or be lost entirely. The FPN neck described in Section 2.4.3 partially mitigates this, but the representation of very small objects remains inherently weaker than that of larger ones [23].

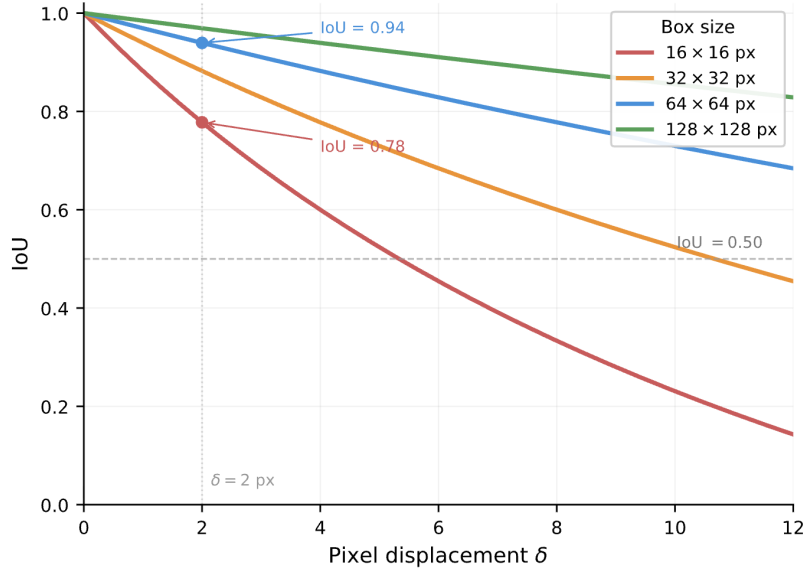


Figure 3.2: Effect of pixel displacement δ on IoU for square bounding boxes of varying size. A single-axis shift is applied to the predicted box while the ground-truth box remains fixed. At $\delta = 2$ pixels, a 16×16 box drops to $\text{IoU} = 0.78$ while a 64×64 box retains $\text{IoU} = 0.94$, illustrating why small-object detection is disproportionately sensitive to localisation error. The dashed line marks the standard mAP_{50} threshold.

3.5.2 Strategies for Small-Object Detection

Several strategies have been proposed to address these difficulties. Multi-scale detection heads, as used in FPN-based architectures, are the most widely adopted architectural response; some methods extend this with additional high-resolution prediction scales or dilated convolutions [23]. At the data level, augmentation techniques such as copy-pasting small object instances into new training locations improve recall on under-represented size categories [19], while inference-time tiling increases effective resolution at the cost of higher computation.

A complementary approach is to train a dedicated detector whose capacity, input resolution, and confidence thresholds are tuned specifically for the small-object class.

This avoids the class-imbalance problem inherent in joint training and allows the operating point to be optimised independently. In particular, operating at a low confidence threshold τ maximises recall at the cost of increased false positives — a trade-off that can be managed by downstream temporal filtering rather than within the detector itself [14].

3.5.3 Temporal Filtering of Ball Candidates

When a detector operates at a low confidence threshold, the resulting candidate set at each frame contains both the true ball and spurious detections. The general form of this set at frame t is

$$C_t = \left\{ (\mathbf{b}_t^{(j)}, s_t^{(j)}) \right\}_{j=1}^{N_t}, \quad (3.1)$$

where $\mathbf{b}_t^{(j)}$ is a candidate location and $s_t^{(j)}$ its confidence score.

Temporal coherence provides a principled basis for disambiguation. Because the ball follows a physically continuous trajectory, its inter-frame displacement is bounded even under rapid motion. Two families of filtering exploit this property. Motion-model approaches, such as the Kalman filter described in Section 2.5.1, predict the ball’s next position using an explicit dynamical model and select the closest candidate. These are effective when the assumed model holds, but can diverge under the abrupt accelerations from kicks, deflections, and rebounds that are routine in football. An alternative family enforces only that inter-frame displacement remains below a physically plausible bound, discarding candidates that would imply impossible ball speeds regardless of confidence. This sacrifices predictive power during smooth motion but is more robust to the sharp trajectory changes characteristic of ball dynamics.

The choice between these families, and the specific instantiation used in this project, is discussed in Chapter 3. Chapter 5 reports ball detection performance separately from the general object detector.

3.5.4 Dedicated Ball Detector Training

The dedicated ball detector was initialised from the pretrained `yolo8x.pt` checkpoint and fine-tuned for 150 epochs at input resolution 1280×1280 , with a batch size of 8, early-stopping patience of 30 epochs, and the Ultralytics `auto` optimiser resolving to AdamW with a learning rate of 2×10^{-3} . Augmentation settings mirrored those used for the generic detector, with `mixup` and `copy_paste` disabled. Training was

carried out on an NVIDIA A100-80GB GPU on the University of Manchester CSF3 HPC facility, with mixed-precision (AMP) enabled throughout, and the full run took approximately 109 minutes of wall-clock time. The full training log and per-epoch `results.csv` are retained under `artifacts/ball_detection_model_yolov8-x/`, allowing the training curves to be reproduced exactly.

Choice of the larger variant. The ball detector uses the `yolov8x` variant (68.1 M parameters) rather than the `yolov8m` variant used for the generic detector. The motivation is that the ball is a small, fast-moving object whose appearance offers very little discriminative signal per pixel, and a higher-capacity backbone is better able to extract useful features from the few pixels on target. Because the dedicated detector runs only once per frame on the same input as the generic detector, and because the ball pipeline does not participate in per-frame tracking, the additional inference cost of the larger variant is acceptable within the overall runtime budget.

3.5.5 Pitch Keypoint Detector Training

The pitch keypoint model was initialised from the pretrained `yolov8x-pose.pt` checkpoint and fine-tuned for 500 epochs at input resolution 640×640 , with a batch size of 48, early-stopping patience of 100 epochs, and the Ultralytics `auto` optimiser. Training was carried out on an NVIDIA A100-40GB GPU in a Google Colab environment, with mixed-precision enabled throughout, and the full run took approximately 45 minutes of wall-clock time. The full training log and per-epoch `results.csv` are retained under `artifacts/pitch_keypoint_detection_model/`.

Mosaic disabled for keypoint training. One departure from the default augmentation used elsewhere is that mosaic augmentation was explicitly disabled. Mosaic composes four images into a single training image by tiling; for bounding-box detection this is helpful (it increases the number of objects seen per batch and encourages context-invariance), but for keypoint tasks the tiling disrupts the geometric relationships between keypoints that the model must learn, since keypoints near the tile boundaries may be cropped or relocated in ways that do not correspond to any real camera viewpoint. Horizontal flip was retained because the keypoint layout defines an explicit left-right symmetry via `flip_idx`; vertical flip and perspective augmentation were disabled for the same geometric-integrity reasons.

Deployment path. The trained weights were exported and deployed to Roboflow for hosted inference, and the pipeline consumes the keypoint model through Roboflow’s hosted inference API rather than loading the local weights directly. The rationale for this deployment choice is explained in Section ??, which discusses the pitch registration stage that consumes this model’s output.

3.6 Detection and Tracking

The first operational stage of the pipeline detects the visual entities required for downstream football analysis and assigns persistent identities to players across frames. This stage is critical because later components, including team identification, pitch-space projection, possession estimation, heatmaps, and offside candidate detection, all depend on the reliability of the detections and tracks produced here.

3.6.1 Main Object Detection

The main detector is a YOLOv8m object detection model trained to recognise four classes: players, goalkeepers, referees, and the ball. During inference, each video frame is passed through the detector at an image size of 1280 pixels. The runtime confidence threshold is set to 0.2 and the IoU threshold is set to 0.5. These values were chosen to retain sufficient detections for the tracker while still removing low-confidence false positives.

The detector output for each frame consists of a set of bounding boxes, class labels, and confidence scores. These detections are converted into `supervision` detection objects so that they can be passed consistently between the object detection, tracking, annotation, and tactical analysis modules. Preserving this intermediate representation also makes it easier to inspect and evaluate the behaviour of individual pipeline stages.

Although the main detector includes a ball class, the ball is substantially harder to detect than players because it occupies very few pixels and is often blurred or occluded. For this reason, the main detector is used primarily for players, goalkeepers, and referees, while ball localisation is strengthened using a separate dedicated detector.

3.6.2 Dedicated Ball Detection and Filtering

A dedicated YOLOv8x ball detector is applied to each frame to improve the reliability of ball localisation. This design decision was made because errors in ball detection

propagate directly into possession, pass, and offside analysis. A missed or incorrect ball location can cause the system to assign possession to the wrong player, miss a pass transition, or trigger an incorrect offside candidate event.

The ball detector is run with a low initial confidence threshold (`ball_conf = 0.05`) so that weak ball candidates are not discarded too early. Candidate detections are then filtered temporally. A ball candidate is accepted only if it is sufficiently confident or spatially consistent with the previous accepted ball position. In particular, the pipeline uses a maximum jump threshold of 80 pixels and a minimum stronger-confidence threshold of 0.25. This prevents isolated low-confidence detections far from the previous ball position from being selected as the ball.

When multiple ball candidates are present, the system selects the candidate that is most consistent with the recent ball position. This filtering strategy does not solve all cases of ball occlusion or fast motion, but it reduces the effect of isolated false positives and produces a more stable ball trajectory for downstream tactical reasoning.

3.6.3 Multi-object Tracking

Player tracking is performed using tracking-by-detection. In each frame, the main YOLO detector supplies bounding boxes to a tracker, which assigns a persistent track identifier to each detected player. The default tracker used in the pipeline is ByteTrack, with BoT-SORT available as an alternative configuration. The tracker is selected through the runtime tracker configuration, allowing the same pipeline to be evaluated with different tracking algorithms.

ByteTrack was selected as the default tracker because it is designed to use both high-confidence and lower-confidence detections during association. This is useful in broadcast football footage, where players are frequently partially occluded and their detection confidence may temporarily fall even when they remain visible. By retaining lower-confidence detections for a second association step, ByteTrack can reduce unnecessary track fragmentation in crowded scenes.

The tracker returns detections with associated track identifiers. These track identifiers are preserved in the pipeline and are used by later stages to accumulate player positions, smooth team labels, generate heatmaps, and compute player-level statistics. Detections without valid track identifiers are not used for player-level temporal statistics, since they cannot be linked reliably across frames.

3.6.4 Track Handling and Failure Cases

The tracking stage is vulnerable to several common failure cases. Occlusion can cause a player to disappear temporarily, leading to a broken track. Similar player appearance can cause identity switches, particularly when players cross or cluster around the ball. Camera motion can also affect the apparent movement of detections and make association more difficult.

The pipeline mitigates these issues in two ways. First, the tracker itself uses temporal association to preserve identities across short interruptions. Second, downstream stages avoid relying on a single frame wherever possible. For example, team labels are smoothed across a fixed track history, and movement statistics include a maximum-speed gate to reduce the effect of spurious projected movements.

Nevertheless, tracking errors cannot be fully eliminated at this stage. A fragmented or incorrect track can affect player heatmaps, distance estimates, team-level possession, and offside candidate detection. For this reason, the evaluation considers both the component-level behaviour of the tracker and the effect of tracking errors on the integrated pipeline.

3.7 Team Identification

After player detections have been assigned track identifiers, the next stage is to infer the team membership of each tracked player. This is required for team-level outputs such as possession statistics, pass events, offside candidate detection, and team-coloured visualisations. The system performs team identification without match-specific manual labels by fitting an unsupervised classifier from player crops sampled from the input video.

3.7.1 Warm-up Sampling and Player Cropping

Team identification begins with a warm-up phase at the start of each video. During this phase, frames are sampled from the first 20 seconds of footage using a stride of 30 frames. The main detector is applied to these sampled frames, and only detections classified as players are retained. Very small crops are discarded using a minimum bounding-box size of 35 by 35 pixels, since such detections usually contain too little visual information for reliable team assignment.

For each retained player detection, the system extracts a torso crop rather than

using the full bounding box. The crop is taken from approximately 15% to 65% of the bounding-box height. This focuses the representation on the shirt region, which usually contains the strongest team-specific visual signal, while reducing noise from shorts, legs, grass, and nearby players. A maximum of 800 warm-up crops is used, and at least 50 crops are required before the classifier is fitted.

3.7.2 Embedding and Clustering

The retained torso crops are used to fit a team classifier. The classifier extracts visual embeddings from each crop using a SigLIP-based image encoder, then reduces the embedding dimensionality using UMAP and clusters the reduced representations using K-means with $k = 2$. The two clusters correspond to the two outfield teams.

This approach was selected because it avoids requiring labelled team examples for every new match. A supervised team classifier would need new annotations whenever kits, lighting conditions, competitions, or broadcast styles changed. By fitting the classifier from the input video itself, the system can adapt to the visual appearance of the teams in the current match.

During the per-frame pipeline, each tracked player crop is passed through the same embedding and clustering pipeline to obtain a predicted team label. The output is a binary team assignment for outfield players. These assignments are then stabilised using temporal smoothing before being used by downstream analytics.

3.7.3 Temporal Smoothing and Role Handling

Frame-level team predictions can fluctuate because of occlusion, motion blur, partial crops, lighting variation, or players turning away from the camera. To reduce this instability, the system applies track-level majority-vote smoothing. For each track identifier, the most recent team predictions are stored in a fixed-length history of 30 predictions. The smoothed team label is then taken as the majority label within this history.

This design reduces the effect of isolated incorrect predictions while still allowing the team label to update if a track was initially misclassified. Smoothing is especially important because team labels are used in later components where a single-frame error can affect possession, pass, or offside logic.

Goalkeepers and referees require separate handling. Referees are assigned to a non-team class because they do not contribute to team possession or offside analysis. Goalkeepers are detected as a separate object class and are assigned to the team whose

outfield-player centroid is nearest in image space. This spatial rule is used because goalkeeper kits often differ from the outfield players' kits, making appearance-based clustering less reliable for this role.

The resulting representation assigns each detection either to team 0, team 1, or a non-team class. This representation is used consistently by the visualisation, possession, pass, and offside modules.

3.8 Pitch Registration

Pitch registration maps image-space detections from the broadcast frame onto a canonical football pitch. This stage is required because image coordinates are affected by camera position, zoom, and perspective distortion. Tactical outputs such as heatmaps, speed, distance, possession proximity, and offside candidate detection therefore need to be computed in pitch space rather than directly in the broadcast image.

3.8.1 Pitch Keypoint Detection

Pitch landmarks are detected using a YOLOv8x-pose model trained to localise 32 canonical pitch keypoints. For each processed frame, the model predicts a set of keypoint coordinates and confidence scores. Keypoints with confidence below 0.5 are discarded before homography estimation. This filtering step reduces the effect of uncertain keypoint predictions, which can otherwise produce unstable or incorrect pitch transformations.

A homography is estimated only when at least six valid keypoints are available. Although four point correspondences are theoretically sufficient to estimate a homography, requiring more than four points makes the transformation less dependent on any single detected landmark. Frames with too few reliable keypoints reuse the most recent valid transformation where possible, rather than forcing an unstable new estimate.

3.8.2 Homography Estimation and Coordinate Transformation

For each valid set of pitch keypoints, the system estimates a homography between the detected image-space landmarks and their corresponding locations on a canonical pitch model. The homography provides a projective transformation from image coordinates to pitch coordinates. Once this transformation is available, detected object positions can be expressed in pitch space.

For players, goalkeepers, and referees, the image-space position is taken as the bottom-centre point of the bounding box, since this approximates the point where the player contacts the ground. For the ball, the centre of the bounding box is used. These points are then transformed into pitch coordinates using the estimated homography.

The pitch coordinate system is represented in centimetres, with a pitch length of 12,000 cm. Using metric-like pitch units allows later stages to express distance thresholds and spatial rules in interpretable terms. For example, the possession and offside modules use distances in centimetres rather than pixel distances, making the thresholds less dependent on camera zoom.

3.8.3 Caching and Robustness Measures

Estimating a new pitch transformation on every frame is computationally expensive and can also introduce visual jitter when keypoint predictions vary slightly between consecutive frames. To reduce this problem, the pipeline recomputes the pitch transformation every 15 frames and caches the most recent valid transformation between updates.

This stride-cached design is a compromise between accuracy, stability, and runtime. Updating the homography too rarely can make the transformation lag behind camera motion, while updating it every frame can amplify keypoint noise. A stride of 15 frames provides regular updates while reducing unnecessary recomputation.

Pitch registration remains one of the most important sources of downstream error. If the homography is inaccurate, then all pitch-space outputs derived from it may be affected, including heatmaps, speed and distance estimates, possession proximity, and offside geometry. For this reason, registration quality is inspected both visually through bird’s-eye outputs and quantitatively where keypoint validation metrics are available.

3.9 Tactical Output Generation

Once detections have been tracked, assigned team labels, and transformed into pitch coordinates, the system generates tactical outputs from the resulting structured representation. These outputs are intended to make the broadcast video more analytically useful by converting low-level detections into visual summaries, player statistics, team possession estimates, pass events, and candidate offside situations.

3.9.1 Bird's-eye Visualisation

The bird's-eye visualisation represents the detected players, goalkeepers, referees, and ball on a canonical top-down pitch. For each frame with a valid pitch transformation, the image-space object positions are projected into pitch coordinates and rendered on the pitch model. Players are coloured according to their inferred team labels, referees are shown as a separate non-team class, and the ball is rendered using its filtered pitch-space position when available.

This visualisation provides an interpretable way to inspect whether the detection, team assignment, and pitch registration stages are behaving coherently. It is also used to generate side-by-side videos, where the original broadcast frame and the corresponding pitch-space representation are displayed together. This makes it easier to identify errors such as incorrect team labels, unstable homographies, or ball locations that do not match the broadcast footage.

3.9.2 Heatmaps, Speed and Distance

Player heatmaps are generated by accumulating each tracked player's pitch-space positions over time. The pitch is divided into a fixed grid of 480×280 cells, and each valid player position contributes to the corresponding grid location. Heatmaps are generated only when at least 40 valid samples are available for a track, reducing the likelihood of producing misleading visualisations from short or fragmented tracks.

To improve readability, the accumulated heatmap is smoothed using a Gaussian blur with kernel size 31 and a gamma correction of 0.22. The final heatmap is saved both as an image and as a raw numerical array, so that the visual output can be inspected while the underlying positional distribution remains available for further analysis.

Speed and distance estimates are computed from changes in player pitch-space position over time. Consecutive valid positions for each track are used to estimate displacement, which is accumulated to produce total distance travelled. Speed is derived from displacement and frame timing. Since small errors in pitch registration can produce unrealistically large apparent jumps, the system applies a maximum-speed gate of 40 km/h. Movements above this threshold are treated as noise and excluded from the accumulated statistics.

3.9.3 Possession and Pass Detection

Possession is estimated using the pitch-space distance between the filtered ball position and nearby players. For each frame, the system identifies the player closest to the ball. A player is considered a possession candidate only if the distance to the ball is below 350 cm. To reduce noise from brief misdetections or transient ball movements, possession must be stable for at least three consecutive frames before being confirmed.

Team possession is then obtained from the confirmed player's team label. If no player satisfies the proximity condition, the frame is treated as having no new confirmed possession event. This conservative design reduces the risk of assigning possession during periods when the ball is airborne, occluded, or incorrectly detected.

Pass events are detected from stable changes in possession. A pass is recorded when confirmed possession changes from one player to another in a temporally stable way. This rule-based definition is intentionally simple: it is designed to identify likely possession transfers rather than to provide a complete semantic classification of football actions. As a result, the pass output is best interpreted as a set of candidate pass events that require evaluation against manually inspected clips.

3.9.4 Offside Candidate Detection

Offside candidate detection is performed in pitch space using the detected players, inferred team labels, ball position, and estimated attacking direction. The attacking direction is inferred automatically from accumulated team x -centroids once at least 30 valid team-position samples are available, rather than being manually specified for each video.

For each frame with sufficient information, the system estimates the offside line from the second-last defender's pitch-space position. If only one defender is available, a fallback line is computed using the available defensive player, although this case is less reliable. An attacker is treated as a candidate offside player if they are beyond the defensive line by more than a 30 cm margin at the relevant moment and are spatially close enough to the ball event context to be considered involved in play.

The output of this module is not intended to replicate full official offside judgement. The official law depends on contextual factors such as the exact moment the ball is played, active involvement, deliberate play by defenders, and body-part eligibility. The system instead detects candidate offside situations under simplified geometric

assumptions. Candidate events are saved as structured JSON records and visualised using an overlay for a fixed display duration of 50 frames, allowing them to be inspected during evaluation.

3.10 Evaluation Methodology

The evaluation was designed to assess both the individual components of the pipeline and the behaviour of the integrated system. This distinction is important because strong component-level performance does not necessarily imply that the final tactical outputs are reliable. For example, a detector may achieve high validation accuracy while still missing the ball in key moments, or a pitch registration model may perform well on isolated frames while producing unstable transformations during camera motion. The evaluation therefore considers not only model-level metrics, but also the extent to which the complete pipeline produces coherent and useful football analysis outputs.

3.10.1 Component-level Evaluation

The first level of evaluation assesses the supervised models used by the pipeline. The main object detector, dedicated ball detector, and pitch keypoint detector are evaluated using their validation or test splits, with metrics appropriate to each task.

For the object detection models, performance is measured using precision, recall, mAP@50, and mAP@50–95. These metrics assess both whether the relevant objects are detected and how accurately their bounding boxes are localised. The dedicated ball detector is evaluated separately because ball localisation is a known bottleneck for downstream possession, pass, and offside analysis.

For the pitch keypoint model, evaluation considers both bounding-box and keypoint performance where available. The keypoint metrics are particularly important because pitch registration depends on the geometric accuracy of the detected landmarks rather than only on whether the pitch itself is detected.

Tracking is evaluated by comparing the stability and continuity of player tracks across representative clips. Where manual annotations are available, track fragmentation, identity switches, and track continuity are assessed directly. Where dense tracking ground truth is not available, the tracker is evaluated through visual inspection, track-count consistency, and its effect on downstream outputs such as heatmaps and distance estimates.

3.10.2 End-to-end Pipeline Evaluation

The second level of evaluation assesses the complete pipeline on broadcast football clips. This evaluation focuses on whether the system produces coherent outputs after all components have been combined. The main outputs considered are annotated broadcast videos, bird’s-eye visualisations, side-by-side videos, heatmaps, statistics files, and offside event records.

End-to-end evaluation is performed using representative clips containing common broadcast conditions, including camera panning, zooming, player occlusion, ball movement, and changes in pitch visibility. The outputs are inspected to assess whether detected entities remain visually consistent, whether team labels are stable, whether pitch-space projections remain aligned with the broadcast view, and whether the generated statistics are plausible.

Where manual labels are produced, tactical outputs are evaluated against manually inspected ground truth. Possession estimates are compared with the observed player or team in control of the ball, pass events are compared with manually identified possession transfers, and offside candidates are compared with visible attacking situations in the clip. This provides a more meaningful assessment of tactical usefulness than model-level metrics alone.

3.10.3 Ablation Studies

Ablation studies are used to evaluate the contribution of key design decisions. Each ablation changes one component or configuration while keeping the rest of the pipeline fixed, allowing the effect of that design choice to be isolated.

The planned ablations are:

- **Dedicated ball detector.** The pipeline is compared with and without the dedicated ball detector to determine whether the specialised model improves possession, pass, and offside outputs.
- **Tracker choice.** ByteTrack and BoT-SORT are compared on the same clips to assess differences in track continuity, identity stability, and downstream statistics.
- **Team-label smoothing.** Raw frame-level team predictions are compared with track-level majority-vote smoothing to assess whether smoothing reduces label instability.

- **Pitch registration stride.** Different homography update intervals are compared to measure the trade-off between runtime, projection stability, and responsiveness to camera motion.
- **Speed filtering.** Distance and speed estimates are compared with and without the maximum-speed gate to determine how much unrealistic movement is introduced by registration noise.

These ablations are included because the project contains several design choices that could otherwise appear arbitrary. Comparing alternatives makes it possible to justify the final configuration and identify which components have the greatest effect on end-to-end reliability.

3.10.4 Runtime Evaluation

Runtime evaluation measures the practical cost of processing broadcast video. The system records total processing time and, where possible, the time spent in major pipeline stages such as detection, ball detection, pitch registration, team assignment, rendering, and tactical output generation.

Runtime is reported as processing time per frame and effective frames per second. Additional comparisons are made for configurations that change runtime substantially, such as different pitch registration strides or optional bird’s-eye and side-by-side rendering. This allows the evaluation to distinguish between methods that improve output quality and methods that are too expensive for practical use.

The aim is not necessarily to achieve real-time performance, since the project prioritises analytical quality and reproducibility. However, runtime remains an important non-functional requirement because a system that is too slow to run on complete clips would have limited practical value.

3.10.5 Error Analysis

Error analysis is used to examine where the pipeline fails and how errors propagate between stages. This is necessary because the final tactical outputs depend on several earlier components. A missed ball detection can affect possession and pass detection; an identity switch can distort heatmaps and distance estimates; an incorrect team label can affect possession and offside logic; and an unstable homography can corrupt all pitch-space measurements.

The evaluation therefore records representative failure cases and groups them by cause. The main categories considered are missed or false detections, ball tracking errors, track fragmentation, team-label switches, pitch registration failures, and tactical inference errors. For each category, the analysis identifies the likely cause, the downstream effect, and any mitigation used in the pipeline.

This error-focused evaluation is important because the project is an integrated system rather than a single model. The objective is not only to report whether the outputs are correct, but also to explain why they succeed or fail under different broadcast conditions. This provides the basis for the critical evaluation in Chapter 5.

3.11 Implementation and Reproducibility

The pipeline was implemented in Python, using a modular code structure so that individual stages could be configured, tested, and replaced independently. The main entry point is `src/track_video_supervision.py`, which parses the runtime configuration and calls the pipeline orchestration code in `src/pipeline/run.py`. This organisation separates high-level execution from the component-specific modules used for detection, team identification, pitch registration, visualisation, statistics, and offside detection.

3.11.1 Software Environment

The implementation uses Python 3.11. The main libraries are Ultralytics for YOLO model training and inference, Supervision for detection handling and video annotation utilities, OpenCV for image processing and video writing, NumPy for numerical operations, scikit-learn and UMAP for clustering-related operations, and Hugging Face Transformers for the SigLIP-based team embedding model. The Roboflow inference tools are used for the pitch model integration, and the Roboflow sports library is used for the team classification pipeline.

The project dependencies are recorded in `requirements.txt`. In addition, the Roboflow sports library is installed as an editable dependency from its source repository. The environment also requires a Roboflow API key and field model identifier, which are specified through environment variables. An example environment file is provided to document the required variables without exposing private credentials.

3.11.2 Configuration and Model Weights

The pipeline is controlled through runtime parameters rather than hard-coded settings. Important configuration values include the detector image size, detection confidence threshold, IoU threshold, tracker type, pitch registration stride, keypoint confidence threshold, ball filtering thresholds, team smoothing window, heatmap settings, and offside detection parameters.

The default inference configuration uses an image size of 1280 pixels, detector confidence threshold of 0.2, IoU threshold of 0.5, and ByteTrack as the tracker. The team classifier samples the first 20 seconds of video with a stride of 30 frames and applies a 30-frame smoothing window to team predictions. Pitch registration is updated every 15 frames, using keypoints with confidence at least 0.5 and requiring at least six valid keypoints before estimating a homography.

The trained model weights are stored separately from the source code. The required weights are the main object detector `models/object_detection/best.pt`, the dedicated ball detector `models/ball_detection/best.pt`, and the pitch keypoint detector `models/pitch_keypoints/best.pt`. To reproduce the full system, these weights must either be supplied with the project artefacts or regenerated using the documented training scripts and notebooks.

3.11.3 Running the Pipeline

The pipeline can be executed from the command line using the provided run scripts. The standard execution path processes an input video, writes an annotated output video, and optionally generates bird's-eye videos, side-by-side videos, heatmaps, statistics files, pitch debug videos, and offside event records. The main generated outputs are written to the `outputs/` directory.

For each input video, the core output is an annotated video showing detections, track identifiers, team labels, the ball marker, and offside overlays where enabled. Optional outputs include a bird's-eye video, a side-by-side broadcast-and-pitch view, JSON statistics, JSON offside-event records, and per-player heatmap files. The statistics output records team possession, passes, player speed estimates, and distance travelled, while the heatmap directory stores both image files and raw numerical arrays.

This output structure was chosen to support both visual inspection and later evaluation. Videos make it possible to inspect whether the integrated pipeline is behaving

plausibly, while JSON and numerical outputs allow quantitative comparison with manual labels and ablation results.

3.11.4 Reproducibility Limitations

Several limitations affect exact reproducibility. First, the trained model weights are external artefacts and are not stored directly inside the source code repository. Reproducing the system therefore requires either access to the trained weights or re-training the models from the documented datasets and training configuration.

Second, some datasets and hosted models are accessed through Roboflow services, so reproduction depends on access to the relevant workspaces, dataset versions, and API credentials. Third, although YOLO training used deterministic settings and a fixed seed where supported, the runtime team classifier is fitted from sampled video crops and may vary if the underlying embedding, UMAP, or clustering behaviour is not fully deterministic.

Finally, the current implementation is primarily an experimental research pipeline rather than a fully packaged software product. It provides scripts, configuration parameters, model artefact locations, and documented outputs, but does not include a complete automated unit-test suite. This limitation is addressed in the evaluation through component-level validation, visual inspection, ablation studies, and end-to-end testing on representative clips.

3.12 Chapter Summary

This chapter described the methodology used to construct the proposed broadcast football video analysis pipeline. The system was designed around a set of functional and non-functional requirements, with the central objective of producing tactical outputs from a single broadcast video without relying on commercial tracking infrastructure.

The methodology used a modular architecture in which object detection, ball detection, multi-object tracking, team identification, pitch registration, and tactical output generation are implemented as separable stages. This structure allows each component to be configured and evaluated independently, while still supporting end-to-end analysis of the complete pipeline.

The chapter also described the datasets, model training procedures, runtime configuration, and reproducibility measures used by the system. Particular attention was

given to design decisions that affect downstream reliability, including the use of a dedicated ball detector, track-level team-label smoothing, stride-cached pitch registration, and pitch-space rules for possession, pass, and offside candidate detection.

The following chapter reports the results produced by this methodology, including model-level validation results, pipeline outputs, tactical statistics, visualisations, and runtime measurements. Chapter 5 then evaluates these results against the requirements and success criteria defined in this chapter.

Chapter 4

Results

4.1 Experimental Setup

4.2 Findings

Chapter 5

Evaluation

5.1 Analysis

5.2 Discussion

5.3 Limitations

Chapter 6

Conclusion

6.1 Summary

6.2 Future Work

Bibliography

- [1] C. C. Aggarwal, A. Hinneburg, and D. A. Keim. On the surprising behavior of distance metrics in high dimensional space. In *International Conference on Database Theory (ICDT)*, pages 420–434. Springer, 2001.
- [2] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky. Bot-sort: Robust associations multi-pedestrian tracking. *arXiv preprint arXiv:2206.14651*, 2022.
- [3] S. Altmann, L. Forcher, L. Ruf, A. Beavan, T. Groß, P. Lussi, and M. Grossniklaus. Match-related physical performance in professional soccer: Position or player specific? *PLOS ONE*, 16(9):e0256695, 2021.
- [4] M. Beetz, S. Gedikli, J. Bandouch, B. Kirchlechner, N. von Hoyningen-Huene, and A. Perzylo. Visually tracking football games based on tv broadcasts. In *Proceedings of the Twentieth International Joint Conference on Artificial Intelligence (IJCAI)*, 2007.
- [5] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016.
- [6] A. Cioppa, S. Giancola, A. Delière, L. Kang, X. Zhou, Z. Cheng, B. Ghanem, and M. Van Droogenbroeck. Soccernet-tracking: Multiple object tracking dataset and benchmark in soccer videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3491–3502, June 2022.
- [7] J. Davis, L. Bransen, L. Devos, A. Jaspers, W. Meert, P. Robberechts, J. Van Haaren, and M. Van Roy. Methodology and evaluation in sports analytics: challenges, approaches, and lessons learned. *Machine Learning*, 113:6977–7010, 2024.

- [8] A. Delière, A. Cioppa, S. Giancola, M. J. Seikavandi, J. V. Dueholm, K. Nasber, B. Ghanem, and M. Van Droogenbroeck. SoccerNet-v2: A dataset and benchmarks for holistic understanding of broadcast soccer videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 4508–4519, 2021.
- [9] P. Dendorfer, A. Osep, A. Milan, K. Schindler, D. Cremers, I. Reid, S. Roth, and L. Leal-Taixe. Motchallenge: A benchmark for single-camera multiple target tracking. *International Journal of Computer Vision*, 129(4):845–881, 2021.
- [10] S. Ellens, D. Hodges, S. McCullagh, J. J. Malone, and M. C. Varley. Interchangeability of player movement variables from different athlete tracking systems in professional soccer. *Science and Medicine in Football*, 6(1):1–6, 2022.
- [11] J. Fernandez and L. Bornn. Wide open spaces: A statistical technique for measuring space creation in professional soccer. In *MIT Sloan Sports Analytics Conference*, 2018.
- [12] F. R. Goes, L. A. Meerhoff, M. J. O. Bueno, D. M. Rodrigues, F. A. Moura, M. S. Brink, M. T. Elferink-Gemser, A. J. Knobbe, S. A. Cunha, R. S. Torres, and K. A. P. M. Lemmink. Unlocking the potential of big data to support tactical performance analysis in professional soccer: A systematic review. *European Journal of Sport Science*, 21(4):481–496, 2021.
- [13] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.
- [14] A. Gualtieri, E. Rampinini, A. Dello Iacono, and M. Beato. High-speed running and sprinting in professional adult soccer: Current thresholds definition, match demands and training strategies. a systematic review. *Frontiers in Sports and Active Living*, 5:1116293, 2023.
- [15] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition, 2003.
- [16] M. Istasse, J. Moreau, and C. De Vleeschouwer. Associating players to teams using visual features trained with weak supervision. In *International Conference on Machine Vision Applications (MVA)*, 2019.
- [17] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8, 2023.

- [18] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- [19] M. Kisantal, Z. Wojna, J. Muber, A. Pirinen, and E. Rahtu. Augmentation for small object detection. In *9th International Conference on Advanced Concepts for Intelligent Vision Systems (ACIVS)*, 2019. arXiv preprint arXiv:1902.07296.
- [20] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 2012.
- [21] H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- [22] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [23] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2117–2125, 2017.
- [24] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755. Springer, 2014.
- [25] D. Linke, D. Link, and M. Lames. Football-specific validity of TRACAB’s optical video tracking systems. *PLOS ONE*, 15(3):e0230179, 2020.
- [26] J. Liu, X. Tong, W. Li, T. Wang, Y. Zhang, and H. Wang. Automatic player detection, labeling and tracking in broadcast soccer video. *Pattern Recognition Letters*, 30(2):103–113, 2009.
- [27] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg. SSD: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, pages 21–37. Springer, 2016.
- [28] L. Lolli, P. Bauer, C. Irving, D. Bonanno, O. Höner, W. Gregson, and V. Di Salvo. Data analytics in the football industry: a survey investigating operational frameworks and practices in professional clubs and national federations from around the world. *Science and Medicine in Football*, 2024.

- [29] J. Luiten, A. Osep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe. A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision*, 129(2):548–578, 2021.
- [30] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297, 1967.
- [31] F. Magera, T. Hoyoux, O. Barnich, and M. Van Droogenbroeck. BroadTrack: Broadcast camera tracking for soccer. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, Tucson, Arizona, USA, February 2025.
- [32] A. Maglo, A. Orcesi, J. Denize, and Q. C. Pham. Individual locating of soccer players from a single moving view. *Sensors*, 23(18):7938, 2023.
- [33] L. McInnes, J. Healy, N. Saul, and L. Großberger. Umap: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29):861, 2018.
- [34] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024.
- [35] K. O’Shea and R. Nash. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.
- [36] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.
- [37] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.
- [38] J. Redmon and A. Farhadi. YOLO9000: Better, faster, stronger. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6517–6525, 2017.

- [39] J. Redmon and A. Farhadi. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- [40] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- [41] V. Somers, V. Joos, A. Cioppa, S. Giancola, S. A. Ghasemzadeh, F. Magera, B. Standaert, A. M. Mansourian, X. Zhou, S. Kasaei, B. Ghanem, A. Alahi, M. Van Droogenbroeck, and C. De Vleeschouwer. Soccernet game state reconstruction: End-to-end athlete tracking and identification on a minimap. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3293–3305, June 2024.
- [42] The International Football Association Board. Laws of the game 2025/26. <https://downloads.theifab.com/downloads/laws-of-the-game-2025-26-single-pages?l=en>, 2025. Accessed 2026-04-08.
- [43] J. Theiner and R. Ewerth. Tvcilib: Camera calibration for sports field registration in soccer. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1166–1175, 2023.
- [44] J. Theiner, W. Gritz, E. Müller-Budack, R. Rein, D. Memmert, and R. Ewerth. Extraction of positional player data from broadcast soccer videos. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1463–1473, 2022.
- [45] F. Vidal-Codina, N. Evans, B. El Fakir, and J. Billingham. Automatic event detection in football using tracking data. *Sports Engineering*, 25(1), 2022.
- [46] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems*, volume 27, 2014.
- [47] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. *arXiv preprint arXiv:2303.15343*, 2023.
- [48] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. Bytetrack: Multi-object tracking by associating every detection box. In *Computer*

Vision – ECCV 2022, volume 13682 of *Lecture Notes in Computer Science*, pages 1–21. Springer, 2022.

Appendix A

Additional Materials